



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Continuous-Time Approximate Dynamic Programming as an Active Learning Problem

Master Thesis

Balduin Dettling[‡]

July 2, 2024

Advisors:

Bhavya Sukhija[†], Lenart Treven[†],
Prof. Dr. Andreas Krause[†], Prof. Dr. Florian Dörfler[‡]

[†] Department of Computer Science

[‡] Department of Information Technology and Electrical Engineering
ETH Zürich

Abstract

Continuous-time optimal control problems are typically addressed with trajectory optimisation, which gives locally optimal solutions. In discrete time, *approximate dynamic programming* is a widely used paradigm to obtain approximate global solutions to optimal control problems.

We propose an approach to approximate dynamic programming in continuous time. It is based on generating a large set of locally optimal trajectories, which are found efficiently by simulating the optimally controlled system in backward time. The question of where such trajectories should be generated is addressed with tools from active learning. We present a method of approximating the value function (representing the globally optimal control law) based on a large dataset of locally optimal trajectories.

In practical experiments involving a 2D and a 6D problem, we observe that our methods compute accurate approximations of the optimal control law in most regions. However, accuracy deteriorates close to discontinuities of the optimal control law, which presents a fundamental limitation of our methods.

Contents

Contents	iii
Table of Symbols	v
1 Introduction	1
1.1 Background	1
1.2 Problem Setting	1
1.3 Related Work	2
1.3.1 Trajectory Optimisation	2
1.3.2 Learning from Trajectory Optimisation	3
1.3.3 Dynamic Programming	4
1.3.4 Level Set Methods	5
1.3.5 PMP Backward Integration	5
2 Fundamentals	7
2.1 Optimal Control	7
2.1.1 Problem Setting	7
2.1.2 Value Function	8
2.1.3 Hamilton-Jacobi-Bellman PDE	8
2.1.4 Pontryagin Minimum Principle	9
2.1.5 Connections between HJB PDE and PMP	10
2.1.6 PMP on Manifolds	12
2.1.7 Problem Assumptions	13
2.2 Active Learning	15
2.2.1 Batched Active Learning	15
2.2.2 Constrained Active Learning	15
2.3 Sobolev Training	16
2.3.1 Sobolev Training on Manifolds	16
2.3.2 Sobolev Training with Uncertainty Estimates	17
3 Proposed Methods	19

3.1	Level-Set Dynamic Programming	19
3.2	Active Learning Formulation	21
3.2.1	Main Idea	21
3.2.2	Algorithm	22
4	Experiment Results	27
4.1	Orbits System	27
4.2	Flatquad System	29
5	Discussion	33
5.1	Discussion of Results	33
5.2	Comparison with Existing Research	34
5.3	Further Research	36
5.4	Conclusion	40
A	Implementation Aspects	41
A.1	LQR Initialisation	41
A.2	Value Level Estimation	42
A.3	Next Value Target	43
A.4	Proposals	43
A.5	Trajectory Generation	45
A.6	Training and Pruning	45
A.7	Solving ODEs on Manifolds	47
A.8	Tangent and Cotangent Projections	48
A.9	Hamiltonian Minimisation Problem	49
B	Problem Definitions	51
B.1	Orbits	51
B.2	Flatquad	51
C	Algorithm Parameters	53
D	Closed-Loop Cost Evaluation	55
E	Trajectory Optimisation Reference Solution	57
F	Parameter Sweeps	59
	Bibliography	65

Table of Symbols

Symbol	Description
$\mathcal{X}$	State space
x	State space point
n	Dimensionality of $\mathcal{X}$
p	Dimensionality of ambient space of $\mathcal{X}$
$\mathcal{U}$	Input set
u	Input vector
n_u	Dimensionality of $\mathcal{U}$
f	Dynamics function $\mathcal{X} \times \mathcal{U} \rightarrow T_x\mathcal{X}$
l	Cost function $\mathcal{X} \times \mathcal{U} \rightarrow [0, \infty)$
∇_z	Total derivative with respect to z
∂_z	Partial derivative with respect to z
$\dot{z}$	Time derivative $\frac{d}{dt}z(t)$
V	Value function $V : \mathcal{X} \rightarrow [0, \infty)$
$V^\Theta = (\mu^\Theta, \sigma^\Theta)$	Parametric approximation of V with parameters Θ
μ^Θ	Mean estimate of value function
σ^Θ	Uncertainty estimate of value function
$\mathcal{V}_k$	Sublevel set of V : $\mathcal{V}_k = \{x : V(x) \leq v_k\}$
$\mathcal{V}_k^\Theta$	Sublevel set of μ^Θ : $\mathcal{V}_k^\Theta = \{x : \mu^\Theta(x) \leq v_k\}$
u_Θ^*	Greedy controller wrt. approximate value function μ^Θ $u_\Theta^*(x) = u^*(x, \nabla_x \mu^\Theta(x))$
V_Θ^{cl}	Closed-loop cost $\int_0^\infty l(x, u)dt$, with $x(0) = x_0, \dot{x} = f(x, u_\Theta^*(x))$

Introduction

1.1 Background

Optimal control presents a general, yet intuitive approach to specifying state-feedback control laws for dynamical systems by selection of a single cost function. However, explicit evaluation of the optimal control input is only possible approximately, with the exception of a few simple cases. Existing approaches involve various tradeoffs – we provide a short overview in 1.3.

1.2 Problem Setting

We consider the continuous-time, infinite-horizon, deterministic optimal control problem for a control-affine dynamical system. We seek a numerical approach to approximately solving it with the following design goals in mind:

- Practical, smooth approximation of the value function V (and thus the globally optimal control law)¹ in some given domain
- Reliable and fast evaluation of the control law at inference time, while shifting extensive computations to an offline phase

To achieve those goals, we separate the problem into two high-level steps: Generating a large dataset of locally optimal trajectories, and subsequently finding an approximation of V based on that data with supervised learning techniques.

¹With our definitions, V describes the optimal control law, and the cost-to-go function of a suboptimal control law is *not* a value function. This is common in control theory literature but different from RL literature, where typically the cost-to-go function of a suboptimal policy is also called a “value function”.

1.3 Related Work

Various approaches exist to approximately solve continuous-time optimal control problems. Here, we review some of them, along with their strengths and weaknesses.

1.3.1 Trajectory Optimisation

One common real-world application of optimal control is in the form of *trajectory optimisation* (TO). This practice consists in explicitly parameterising a trajectory over some time horizon, and subsequently using an optimisation algorithm to find a trajectory which is (typically locally) optimal with respect to the chosen cost function.

A review of different approaches of TO can be found in [1]. One main distinction is between *direct* and *indirect* methods: While direct methods first discretise the continuous-time problem, approximating it with a suitable discrete-time problem, indirect methods work directly with the continuous-time optimality conditions.

Despite the time discretisation, direct methods can still obtain very accurate approximations of the continuous-time solution. Notably, collocation methods [1, 2] parameterise the input and state trajectory as a polynomial over each time segment. Similarly to ODE solvers, the order of this polynomial determines the order of convergence to the continuous-time solution with increasing time resolution [3].

TO may be used to construct a state feedback controller, defined for each state by finding a locally optimal trajectory starting at that state, and applying the first input of the optimal input sequence. This is a widely used practice known as *Model predictive control* (MPC). There exists a rich body of literature about MPC methods, characterising stability and (sub-)optimality in various settings such as model mismatch, stochasticity, or suboptimal numerical solutions of the underlying optimal control problem. For an overview, we refer to [4].

The main challenge concerning TO is nonconvexity of the optimisation problem. Local solvers rely on a suitably chosen initial guess to find a good local optimum, or to even find one at all. Sampling based methods such as model predictive path integral control [5] or cross-entropy methods [6] present an alternative that in principle can find the global optimum. However, they are associated with large computational cost and limited scalability to challenging systems. These limitations are particularly problematic for real-time control of fast systems with limited computational resources, such as robotic systems or unmanned aerial vehicles.

1.3.2 Learning from Trajectory Optimisation

Various research efforts have aimed at using TO to generate a dataset of optimal trajectories, to then extract an approximation of the optimal feedback law with supervised learning techniques.

Many previous works [7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20] present approaches to generating large amounts of optimal trajectories, and then approximating the value function (or control law) with either a neural network or sparse polynomial regression [15]. However, those publications do not address the situation of distinct local solutions leading to a discontinuous optimal policy – either they do not show any such experiments, or consider the trajectory optimiser a deterministic function of the initial state², or they rule it out by assuming a continuously differentiable value function.

[21] addresses discrete-time problems, and proposes a particular method of detecting conflicting results arising from local trajectory optimisation. However, instead of finding an approximation of the global optimum for each initial state, they find an approximation of a particular local solution and remove conflicting samples, even ones that correspond to better solutions. This may be more practical in some cases but does not correspond to our stated goal of approximating the globally optimal solution.

[22, 23] present approaches to removing suboptimal data from a large dataset of locally optimal trajectories. However, rather than learning an explicit control law, they use the generated data to initialise a local optimiser at inference time.

[24] identifies a two-dimensional optimal control problem whose solution is not only nonsmooth but discontinuous. The authors employ a heuristic to decompose the value function into individual smooth parts.

Instead of approximating a policy with a continuous function, more advanced paradigms have been proposed that handle discontinuity (in the deterministic setting) or multimodality (in the stochastic setting) more gracefully. In [25, 26, 27], *Mixture of Experts* models are used to reflect the piecewise smooth structure of the optimal control law, allowing for accurate representation of discontinuities. However, they require an additional clustering step before approximating the control law.

Learning not a single input, but a *distribution* over inputs also enables practical representation of distinct local solutions. Examples include [28, 29, 30, 31, 32, 33]. However, these methods are associated with computationally intensive inference, and have so far mostly been applied to the discrete-time, stochastic and discounted setting.

²When in reality it also depends on the initial guess trajectory

1.3.3 Dynamic Programming

Discrete Time Dynamic Programming [34] is the basic principle describing the solution of discrete-time, stochastic optimal control problems. It can be used to precisely solve problems where it is possible to exhaustively enumerate the state and input space.

However, more challenging problems can only be addressed by making suitable approximations. This gives rise to *approximate dynamic programming* (ADP), where usually the value function is approximated with some parametric or nonparametric function approximator.

Continuous Time The continuous time analogue of the Bellman Equation is the Hamilton-Jacobi-Bellman partial differential equation (HJB PDE) [35, Eq. 5.19]. Various approaches exist for approximately solving it in practice, some of which we review here.

Various publications explore solving the HJB equation directly with using Physics-Informed Neural Networks (PINNs) [36]. This methodology consists in finding NN parameters that minimise a loss related to dissatisfaction of a PDE, such as the HJB equation. Under suitable conditions, this leads to the NN being an approximation of the PDE solution. Examples of such works include [37, 38, 39, 40, 41, 42]³. However, the experiments shown in those publications are limited to 2D problems, or higher-dimensional ones where both problem and solution are rotationally symmetric in all directions, and without distinct local solutions leading to a nonsmooth value function [41, 42].

The HJB equation can be equivalently reformulated as an infinite-dimensional linear program, whose decision variable is a function on the state space [43, 44, 45]. This is known as the *dual* or *weak* formulation of the optimal control problem. Such LPs can be addressed by *Sums-of-Squares*, which enables exact consideration of the infinite-dimensional constraints if the problem takes a suitable polynomial form. However, practically it is difficult to achieve high precision solutions with this approach, as pointed out in the conclusion of [43].

[46] presents NN architectures which can *exactly* represent the viscosity solutions of specific HJB equations, and thus the global solution of the associated optimal control problems. However, only very simple examples are shown and it is unclear whether similar exact representations can be found for general, nontrivial control problems.

³We remark that [37] was published before the term PINN was introduced, but their methodology fits the definition given in [36].

1.3.4 Level Set Methods

Level-set methods are an established approach for numerically solving the *Eikonal equation*, which is a special case of the HJB equation when applied to continuous shortest-path problems – see [47, 48].

Level sets are often used in algorithms for numerically addressing reachability problems in dynamical systems and differential games [49, 50, 51, 52, 53, 54]. Level sets of neural networks specifically have also been used extensively and as a tool for representing 3D shapes in computer graphics [55, 56].

1.3.5 PMP Backward Integration

The PMP (2.7) provides local optimality conditions for the optimal control problem we consider in the form of an ODE. We will see that by solving it in backward time, we can directly find a locally optimal trajectory, without any kind of iterative optimisation.

Various researchers have explored this idea previously. It has been used to solve 2D pathfinding problems [57, 58, 59, 60, 61, 62], and optimal control problems [63, 64, 65, 66, 67, 68]. Other names for it include *Euler-Lagrange backward integration* [64] and the *stable manifold method* [67, 68]. However, so far all of those examples are limited to two- or three-dimensional state spaces.

Chapter 2

Fundamentals

2.1 Optimal Control

2.1.1 Problem Setting

Consider the deterministic, continuous-time dynamical system

$$\dot{x} = f(x, u) \quad (2.1)$$

with state space $\mathcal{X} \subseteq \mathbb{R}^n$, a set of feasible inputs $\mathcal{U} \subseteq \mathbb{R}^{n_u}$, and dynamics $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathbb{R}^n$.

We consider the associated deterministic, infinite-horizon, continuous-time, undiscounted, and time-invariant optimal control problem. That is, we aim to solve the following optimisation problem:

$$\min_{\substack{u: [0, \infty) \rightarrow \mathcal{U} \\ x: [0, \infty) \rightarrow \mathcal{X}}} \int_0^\infty l(x(t), u(t)) dt \quad (2.2a)$$

$$\text{s.t.: } \dot{x} = f(x(t), u(t)) \quad \forall t > 0 \quad (2.2b)$$

$$x(0) = x_0 \quad (2.2c)$$

The cost function $l : \mathcal{X} \times \mathcal{U} \rightarrow [0, \infty)$ is chosen by the user to influence the behaviour of the resulting control law. We list precise assumptions ensuring a well-posed problem in 2.1.7.

The solution of this problem has been extensively studied in optimal control theory, of which we provide a short overview in the following sections.

2.1.2 Value Function

One central object in optimal control theory is the *value function* $V : \mathcal{X} \rightarrow [0, \infty)$. It is defined as the pointwise optimal cost-to-go, i.e. the cost term $\int_0^\infty l(x, u)dt$ in the optimal control problem (2.2) evaluated at its solution [35, Eq. 5.2].

If the value function is known, the optimal input at each point x is given by

$$u^*(x, \lambda) = \underset{u \in \mathcal{U}}{\operatorname{argmin}} H(x, u, \lambda) \quad (2.3)$$

where $\lambda = \nabla_x V(x)$ is the *costate*, and $H : \mathcal{X} \times \mathcal{U} \times \mathbb{R}^n \rightarrow \mathbb{R}$ is the *Hamiltonian*, given by:

$$H(x, u, \lambda) := l(x, u) + \langle \lambda, f(x, u) \rangle \quad (2.4)$$

The optimal feedback control law is thus $x \mapsto u^*(x, V_x(x))$, and we will sometimes use the overloaded notation $u^*(x)$ to denote the optimal feedback law.

Two central results in optimal control theory characterise the value function and the optimal trajectories associated with it: The *Hamilton-Jacobi-Bellman partial differential equation* (HJB PDE) and Lev Pontryagin's *Minimum Principle* (PMP). We provide a short summary of each.

2.1.3 Hamilton-Jacobi-Bellman PDE

The HJB PDE provides necessary and sufficient conditions for *global* optimality of the value function V . For our particular version of the problem¹, it takes this form [35, Eq. 5.19]:

$$\min_{u \in \mathcal{U}} H(x, u, V_x(x)) = 0 \quad \forall x \in \mathcal{X} \quad (2.5a)$$

$$V(x_{\text{eq}}) = 0 \quad (2.5b)$$

With H being the Hamiltonian from (2.4), and x_{eq} being the goal state which we want to stabilise. We assume that x_{eq} is an equilibrium of (2.1) with input u_{eq} , and that $(x_{\text{eq}}, u_{\text{eq}})$ is the unique global minimum of l with cost 0 (see Assumption 2).

¹Note that as a consequence of solving the infinite-horizon problem with time-invariant problem data, we obtain a PDE and a solution which are both independent of time.

This PDE generally has no solution in the classical sense. Instead, the concept of *viscosity solutions* [69, Section 10.1] [70] has proven to appropriately characterise its solutions. This involves defining a family of *viscous* approximations of the PDE by adding an extra viscosity term, which do admit classical solutions. The viscosity solution of the original PDE is then defined to be the limit of the solutions of the viscous approximations, as the strength of the viscosity term approaches zero.

Such solutions are nonsmooth with V continuous, but not continuously differentiable². The points where V is nonsmooth correspond to the intersection of different local solutions and will be discussed from various viewpoints in the following sections.

2.1.4 Pontryagin Minimum Principle

The PMP, on the other hand, is a *necessary* condition for optimality of a particular trajectory. It describes also locally optimal, but globally suboptimal trajectories, hence it is not a *sufficient* condition for optimality. Originally presented in [71], it has since been restated to apply in various slightly different settings. We state here the version given in [63]:

Theorem 2.1 (Pontryagin Minimum Principle) *Suppose that $x : [0, \infty) \rightarrow \mathcal{X}$ and $u : [0, \infty) \rightarrow \mathcal{U}$ are locally optimal with respect to (2.2). Then there exists a corresponding costate trajectory $\lambda : [0, \infty) \rightarrow \mathbb{R}^n$, such that the following boundary value problem is satisfied:*

$$\dot{x} = \partial_\lambda H(x, u^*(x, \lambda), \lambda), \quad x(0) = x_0, \quad \lim_{t \rightarrow \infty} x(t) = x_{eq} \quad (2.6a)$$

$$\dot{\lambda} = -\partial_x H(x, u^*(x, \lambda), \lambda) \quad (2.6b)$$

PMP Backward Integration Usually, the PMP is presented with split boundary conditions imposing a specific initial and terminal state, or an initial state and a terminal costate (arising from a terminal value function). This is most natural when there are specific initial and terminal states which we seek to connect with an optimal trajectory. Solving the associated boundary value problem (BVP) numerically presents a particular variant of *trajectory optimisation* (1.3.1).

Instead, we choose to impose all boundary conditions at some terminal time $t_f \in \mathbb{R}$, assuming knowledge of a terminal value function V_f . We include the *local cost-to-go* $v(t) := \int_t^\infty l(x, u)dt$ in the ODE state, which will be useful to calculate as well:

² V may in general also be discontinuous, but we rule this case out with Assumption 5.

$$\dot{x} = \partial_\lambda H(x, u^*(x, \lambda), \lambda), \quad x(t_f) = x_f \quad (2.7a)$$

$$\dot{\lambda} = -\partial_x H(x, u^*(x, \lambda), \lambda), \quad \lambda(t_f) = \nabla_x V_f(x_f) \quad (2.7b)$$

$$\dot{v} = -l(x, u^*(x, \lambda)), \quad v(t_f) = V_f(x_f) \quad (2.7c)$$

This can be numerically solved as a standard initial value problem in backward time. In contrast to using a BVP solver with (2.6), we immediately obtain a locally optimal trajectory ending at x_f , without any kind of iterative optimisation or root-finding. The downside is that we do not know in advance which states will be visited by the trajectory. However, for our goal of solving the control problem for *all* initial states we consider this a good starting point.

Sensitivity Problem The PMP suffers from the well known *sensitivity problem* [72, 73, 74]. In short, the solutions of (2.6) and (2.7) have a sensitivity (w.r.t. any boundary conditions) which grows exponentially with the time horizon over which the ODE is solved.

Intuitively, the sensitivity problem can be understood as follows: The locally optimally controlled system is stable in forward time, and therefore unstable in backward time. This instability naturally leads to sensitivity w.r.t. boundary conditions increasing exponentially over time.

Due to this sensitivity, practical usage of the PMP is limited to short time horizons. While it is possible to solve the ODE over an arbitrarily long time horizon, the associated trajectory can generally be so sensitive w.r.t. the boundary conditions that we have no meaningful control over it by selecting boundary conditions.

2.1.5 Connections between HJB PDE and PMP

Stable manifold Consider the set $W^s := \{(x_0, \lambda_0) \in \mathcal{X} \times \mathbb{R}^n : F(x_0, \lambda_0) = 0\}$, where $F(x_0, \lambda_0) := \lim_{t \rightarrow \infty} x(t)$, with x and λ being the integral curve of (2.6) with initial conditions $x(0) = x_0$ and $\lambda(0) = \lambda_0$. W^s is the zero level set of a smooth function, and therefore it is a smooth submanifold of $\mathcal{X} \times \mathbb{R}^n$ [75, Theorem 5.12]. This set is also known as the *stable manifold* [76, Section 2.2] [63, 64] of (2.6). This set describes *all* locally optimal trajectories of the problem (2.2) including the globally optimal ones, and is therefore very interesting.

Let us define the *local cost-to-go* as a function of (x_0, λ_0) defined by first finding the solution (x, λ) of (2.6) with initial condition (x_0, λ_0) at $t = 0$, and then evaluating the cost-to-go from (x_0, λ_0) as $v(x_0, \lambda_0) := \int_0^\infty l(x, u^*(x, \lambda)) dt$.

Because W^s describes all locally optimal trajectories of the system, we can represent the value function V with:

$$V(x_0) = \min_{(x_0, \lambda_0) \in W^s} v(x_0, \lambda_0) \quad (2.8a)$$

$$\text{s.t.: } x = x_0 \quad (2.8b)$$

This representation also appears in [77, Proposition 2] and amounts to selecting the best among all possible local solutions.

Method of Characteristics It is pointed out e.g. in [35, Section 7.2.2], and [78, 70, 77] that the PMP corresponds to the *method of characteristics* [69, Section 3.2] applied to the HJB PDE. The method of characteristics is a process of converting a PDE to a family of ODEs, whose solutions describe the PDE solution along their trajectories. When applying the method of characteristics to any PDE, two difficulties may arise: *Rarefactions* and *Shocks* [69, 78].

Rarefactions arise when some regions (in which the PDE has a solution) are not reached by any solutions of the characteristic ODE. This is not an issue in our setting: Due to Assumption 4 the optimal control law is defined everywhere, and we may find it at any particular point as a solution of (2.7c), which is well-behaved thanks to Assumption 1 and 3.

Shocks correspond to points where different characteristic curves intersect, giving conflicting information about the solution. At these points, additional information is needed to uniquely define the solution.

In the case of optimal control and the HJB equation, resolving this conflict appears simple – wherever several characteristic curves intersect, choose the one with lowest cost-to-go as the global optimum. However, defining the concept of *viscosity solutions* [69, 70] which appropriately captures this intuition is far from trivial.

Nonsmooth Points From these different viewpoints, we see that in our problem setting, the points where several distinct, locally optimal trajectories are all globally optimal correspond exactly to the nonsmooth points of V .

Distinct locally optimal trajectories are a feature shared by many practical nonlinear optimal control problems. Therefore, nonsmooth V and a discontinuous optimal control law are just as common. As we seek a smooth approximation of the value function, we can expect the points where V is nonsmooth to pose practical challenges and have to pay special attention to them.

These points have been investigated in previous research under various names. In [79] and [80], they are termed *points of indifference*, *Sethi-Skiba*

points or *DNSS points* (after four researchers Dechert, Nishimura, Sethi, and Skiba who worked on such points in the 1970s). In [63], they are called *self-intersecting isoclines*. In [64], the terms *watershed points* and *watershed curve* is used. [24] identifies these points in the classic pendulum swingup task, calling them *the nonsmooth line*. In this work, we use the term *nonsmooth point* for its simplicity and descriptiveness.

A similar concept in the context of geodesic curves on manifolds (which are solutions of a particular optimal control problem) is known under the names “Maxwell point” and “Maxwell set” [81, 82].

In reinforcement learning literature, similar concepts are labelled *policy bifurcation* [27] or *multimodality* [32, 33]. We point out that in this context, multimodality refers to a multimodal action distribution of a stochastic policy. In the deterministic setting, the optimal control input is unique almost everywhere³. Therefore, the policy cannot be considered multimodal in the sense that it is described by a multimodal distribution over inputs.

2.1.6 PMP on Manifolds

Most introductory textbooks on optimal control present the PMP and other standard results for Euclidean state spaces $\mathcal{X} = \mathbb{R}^n$, and up to this point we have done the same. However, many real-world dynamical systems have state variables which evolve on a smooth manifold. In this section, we explain how the PMP for $\mathbb{R}^n$ can be used to solve problems on a manifold.

We choose to represent the state space as an n -dimensional submanifold embedded in the ambient space $\mathbb{R}^p$. The Whitney embedding theorem [75, Chapter 6] states that such an embedding exists for any smooth manifold, so this choice is without loss of generality. We can then consider the control problem as one defined on the ambient space, with a few technicalities which are outlined here. For this exposition, we mostly follow [83].

At the heart of both the PMP and the HJB equation, we have the inner product $\langle \lambda, \dot{x} \rangle$ in (2.4). In the $\mathbb{R}^n$ setting, the costate λ is the gradient of the value function.

When the state space is a smooth submanifold $\mathcal{M} \subset \mathbb{R}^p$, the value function has no gradient, but a differential $dV(x)$ [75, Chapter 3], which is a linear map on the tangent space $T_x\mathcal{M}$ [75, Chapter 5] and thus member of the cotangent space $T_x^*\mathcal{M}$ [75, Chapter 11]. Therefore we should expect a costate $\lambda \in T_x^*\mathcal{M}$.

³When several distinct trajectories are globally optimal, we may arbitrarily choose one of them without losing optimality. This occurs precisely at the points where V is nonsmooth, which form a set of measure zero.

However, when we carry out calculations in the ambient space, we obtain a different costate which we call the *ambient costate*: $\lambda_a \in T_x^*\mathbb{R}^p$. In the following we explore how the ambient costate relates to the costate $\lambda \in T_x^*\mathcal{M}$ which we are interested in.

At some $x \in \mathcal{M}$, consider the tangent space $T_x\mathcal{M}$ as a linear subspace of $T_x\mathbb{R}^p$. Its orthogonal complement (with respect to $T_x\mathbb{R}^p$) is the *normal space* $N_x\mathcal{M}$. We can thus decompose any $v \in T_x\mathbb{R}^p$ as $v = \Pi_{T_x\mathcal{M}}v + \Pi_{N_x\mathcal{M}}v$ (where $\Pi_V : W \rightarrow W$ denotes orthogonal projection onto a linear subspace V of W). The ambient costate λ_a can be similarly decomposed into two orthogonal components which we call $\lambda_t = \Pi_{T_x\mathcal{M}}\lambda_a$ and $\lambda_n = \Pi_{N_x\mathcal{M}}\lambda_a$.⁴

We insert that decomposition into the inner product $\langle \lambda_a, \dot{x} \rangle = \langle \lambda_t, \dot{x} \rangle + \langle \lambda_n, \dot{x} \rangle$, with $\dot{x} \in T_x\mathcal{M}$. Expanding the second term, we find:

$$\langle \lambda_n, \dot{x} \rangle = \langle \Pi_{N_x\mathcal{M}}\lambda, \dot{x} \rangle = \langle \lambda, \Pi_{N_x\mathcal{M}}\dot{x} \rangle = \langle \lambda, 0 \rangle = 0 \quad (2.9)$$

This means that while two ambient costates $\lambda_1, \lambda_2 \in T_x^*\mathbb{R}^p$ with $\lambda_1 - \lambda_2 \in N_x\mathcal{M}$ may represent different linear maps on the domain $T_x\mathbb{R}^p$, they correspond to the same linear map when restricted to the domain $T_x\mathcal{M}$.⁵ Because the costate is only ever used in the inner product $\langle \lambda, \dot{x} \rangle$ with $\dot{x} \in T_x\mathcal{M}$, we can ignore the normal component λ_n .

This motivates us to generally refer to the costate variable as just λ , without explicitly distinguishing between $\lambda_a \in T_x^*\mathbb{R}^p$ or $\lambda_t \in T_x^*\mathcal{M}$ unless necessary. In general, we will reason about control problems on $\mathbb{R}^n$, remembering that the manifold case is addressed as well by choosing an embedding in a higher dimensional ambient space. We will explicitly mention the appropriate restrictions, projections, and other concepts from differential geometry only when necessary.

We additionally assume that we have a smooth function $m : \mathbb{R}^p \rightarrow \mathbb{R}^{p-n}$ which describes the state space as its zero level set: $\mathcal{M} = \{x \in \mathbb{R}^p : m(x) = 0\}$. For the mechanical systems we deal with this is often a very natural description, and it will turn out to be useful for explicitly constructing the projections onto tangent and cotangent space.

2.1.7 Problem Assumptions

In order to ensure that the system (2.1) and the optimal control problem (2.2) have well-defined solutions, we make the following regularity assumptions.

⁴The costate projections can be defined as the adjoint of the projections on tangent and normal space: $\langle \Pi_{T_x\mathcal{M}}\lambda, \dot{x} \rangle = \langle \lambda, \Pi_{T_x\mathcal{M}}\dot{x} \rangle$ and $\langle \Pi_{N_x\mathcal{M}}\lambda, \dot{x} \rangle = \langle \lambda, \Pi_{N_x\mathcal{M}}\dot{x} \rangle$, for $\lambda \in T_x^*\mathbb{R}^p$ and $\dot{x} \in T_x\mathbb{R}^p$. Because they are *orthogonal* projections, they are self-adjoint, and thus we have $\Pi_{T_x\mathcal{M}}^* = \Pi_{T_x\mathcal{M}}$ and $\Pi_{N_x\mathcal{M}}^* = \Pi_{N_x\mathcal{M}}$.

⁵This argument is made more rigorously in [83, Lemma 1].

Assumption 1 In order to guarantee existence and uniqueness of solutions to (2.1) via Picard-Lindelöf, we adopt the following assumptions from [35]:

- f is $\mathcal{C}^1$ in x and continuous in u .
- $\nabla_x f$ is continuous in t and u
- u is a piecewise continuous function of t . □

Assumption 2 In order to guarantee existence of the cost integral in (2.2) and well-posedness of the LQR problem at the equilibrium (A.1), we assume:

- l is $\mathcal{C}^2$ in both arguments
- $l(x_{\text{eq}}, u_{\text{eq}}) = 0$, and $l(x, u) > 0$ for all other $(x, u) \in \mathcal{X} \times \mathcal{U}$.
- $0 = \nabla_x l(x_{\text{eq}}, u_{\text{eq}})$ and $0 = \nabla_u l(x_{\text{eq}}, u_{\text{eq}})$
- The Hessians $Q = \nabla_x^2 l(x_{\text{eq}}, u_{\text{eq}})$ and $R = \nabla_u^2 l(x_{\text{eq}}, u_{\text{eq}})$ are both strictly positive definite □

Assumption 3 To similarly guarantee existence and uniqueness of the solutions of (2.7), we assume the following:

- The function $(x, \lambda) \mapsto H_x(x, u^*(x, \lambda), \lambda)$ is Lipschitz.
- The function $(x, \lambda) \mapsto -H_\lambda(x, u^*(x, \lambda), \lambda)$ is Lipschitz. □

Assumption 4 We assume that:

- The entire state space is stabilisable, that is, for every $x_0 \in X$ there exists some trajectory $x : [0, \infty) \rightarrow \mathcal{X}$ given by the input $u : [0, \infty) \rightarrow \mathcal{U}$ with $x(0) = x_0$ and $\lim_{t \rightarrow \infty} x(t) = x_{\text{eq}}$.
- l is *coercive* in x , i.e. $\lim_{\|x\| \rightarrow \infty} l(x, u) = \infty$ for each $u \in \mathcal{U}$
- $\mathcal{U}$ is compact. □

This ensures existence of an optimal trajectory with respect to (2.2) at all $x_0 \in \mathcal{X}$ [84]. We note that the same can be ensured with unbounded $\mathcal{U}$, if l is also coercive in u .

We make two additional assumptions that narrow down the class of problems we consider, so that we are able to practically find approximate solutions.

Assumption 5 We assume continuity of the value function V , so that we can address the difficulty of a discontinuous value gradient in isolation. One set of assumptions ensuring continuous V is given in [77]. □

Assumption 6 Additionally, we assume the following, such that (2.3) can be evaluated by solving a strictly convex quadratic program (QP):

- f is *control-affine*, i.e. affine in u for each fixed x

- l is quadratic and strictly convex in u
- $\mathcal{U}$ is a *convex polytope*: $\mathcal{U} = \{u \in \mathbb{R}^{n_u} : Gu \leq h\}$ for suitable G, h $\square$

We note that these can be replaced by any other suitable set of assumptions that makes (2.3) practically solvable in some different way.

2.2 Active Learning

Active learning is concerned with machine learning methods which select the points for which training labels will be supplied [85]. In comparison to classical supervised learning, this can lead to substantial improvements in sample efficiency.

Most active learning research is focussed on *greedy* strategies, i.e. iterative selection of an individual input for labelling, followed by updating the model with the new data.

Typically, this is done with an *acquisition function* f_{acq} – a function which quantifies the usefulness of adding a new training datapoint at a specific location. f_{acq} may be maximised to find the “most useful” sample to label next.

2.2.1 Batched Active Learning

Conditioning the model on new data can require considerable computation – in that case it can be desirable to add not just a single sample, but rather an entire batch of $N_{\text{batch}} \in \mathbb{N}$ samples before updating the model [86, 87, 88, 89]. In this case, some mechanism for ensuring diversity of the batch is needed.

One approach for addressing this problem setting is to mimic the iterative greedy selection common in non-batched active learning, but *within* the batch. Rather than updating the full model, a proxy can be used to approximate the uncertainty obtained by adding each new data point. The proxy uncertainty gives rise to an updated proxy acquisition function, which we may maximise to obtain a new proposal.[89] presents an overview of various such methods.

2.2.2 Constrained Active Learning

While traditional active learning methods have the goal of accurately approximating a function for all inputs within some given domain, *constrained* active learning addresses the more challenging situation where constraints on the input have to simultaneously be learned and satisfied.

While most often introduced for the purpose of *Bayesian optimisation* [90, 91, 92], these results mostly transfer to the active learning setting when replacing acquisition functions aimed at finding plausible maximisers (like upper

confidence bound, expected improvement or probability of improvement) by ones purely focussed on exploration (such as pure uncertainty or expected information gain).

2.3 Sobolev Training

When solving (2.7) numerically, we obtain evaluations of both the value function $V(x)$ and its gradient $\lambda(x) = \nabla_x V(x)$. Based on this information, we want to find an approximate value function which accurately represents *both* the value and its gradient. Both pieces of information are needed - the value V for deciding which local solution is the globally optimal one, and the costate $\lambda = \nabla_x V$ because the optimal input u^* depends on it (Eq. 2.3).

Specifically, suppose we have a neural network (NN) with weights $\theta \in \mathbb{R}^{n_\theta}$, which defines a function $V_\theta : \mathbb{R}^n \rightarrow \mathbb{R}$. We want it to be a good approximation of both function labels $V_\theta(x) \approx V(x)$ and gradient labels $\nabla_x V_\theta(x) \approx \nabla_x V(x)$. Therefore, instead of using a standard loss which purely penalises error in function values, we use a loss function which also penalises the gradient error with suitably chosen l_0 and l_1 :

$$l(\theta, x) = l_0(V_\theta(x), V(x)) + l_1(\nabla_x V_\theta(x), \nabla_x V(x)) \quad (2.10)$$

This practice is known as *Sobolev training*. While that term was introduced relatively recently by [93], the same idea has existed previously under a variety of names, such as *double backpropagation* [94], *value-jacobian networks* [95], *physics-informed machine learning* or *gradient regularisation* [9], *deep differential networks* [38], or *gradient learning* [10]. A corresponding universal approximation theorem is proven in [96]. A similar concept for Gaussian processes is also widely used – we refer to [97, Chapter 9.4] and [98].

For the specific task of representing the value function in optimal control with a neural network, [9] and [10] report that Sobolev training with both value and costate information outperforms a similar model trained with either one of the two. The fact that the costate is the gradient of a potential function V acts as a useful inductive bias to narrow down the class of possible functions, and [93] points out that training with evaluations of the potential (V) can also improve the predictive accuracy of the gradient (V_x).

2.3.1 Sobolev Training on Manifolds

In [93] and related works, Sobolev learning is presented as a method for approximating functions on $\mathbb{R}^p$. However, we also deal with systems whose state space is a smooth embedded submanifold $\mathcal{X} \subset \mathbb{R}^p$ (see 2.1.6).

The NN-approximated value function V_θ is defined on the entire ambient space $\mathbb{R}^p$, but we are only interested in the restriction $V_\theta|_{\mathcal{X}}$ closely approximating the actual solution V . For fitting function values, we can ignore this distinction and treat V_θ as a function on $\mathbb{R}^p$, considering that all training data is in $\mathcal{X}$.

For the gradient labels, we need to be more particular. Each training label is an *ambient costate* $\lambda_a \in T_x^* \mathbb{R}^p$. As such it is a linear map on the domain $T_x \mathbb{R}^p$, just like the gradient of the NN approximation $\nabla_x V_\theta(x)$.

However, the value function V is only defined on $\mathcal{X}$, so it has no gradient, but a differential $dV(x) \in T_x^* \mathcal{X}$. Correspondingly, we only require that the restriction $\nabla_x V_\theta|_{T_x \mathcal{X}}$ matches the restriction $\lambda_a|_{T_x \mathcal{X}}$.

As seen in 2.1.6, we can decompose any $\lambda_a \in T_x^* \mathbb{R}^p$ into orthogonal tangent and normal components. The tangent component preserves the restriction onto the tangent space, while the normal component describes a linear map mapping the entire tangent space to 0. We thus only penalise the relevant tangent component of the gradient error, by including the projection defined in 2.1.6 in l_1 :

$$l(\theta, x) = l_0(V_\theta(x), V(x)) + l_1(\Pi_{T_x^* \mathcal{X}} \nabla_x V_\theta(x), \Pi_{T_x^* \mathcal{X}} \lambda_a) \quad (2.11)$$

If the state space is Euclidean, we can still view it as a submanifold of $\mathbb{R}^p$ with $p = n$ (so the submanifold has codimension 0), in which case the projection $\Pi_{T_x^* \mathcal{M}}$ becomes the identity map. We therefore always use (2.11) rather than (2.10) to avoid unnecessary case distinctions.

2.3.2 Sobolev Training with Uncertainty Estimates

In order to give uncertainty estimates, we adopt the simple approach of training an ensemble of N_{ens} neural networks. Each member of the ensemble is a standard, fully connected neural network – specific details about the NN can be found in Appendix C. Randomness enters through a Pseudorandom number generator (PRNG) seed, which is different for each ensemble member and affects both parameter initialisation and minibatch selection.

Any such training process defines a probability distribution over NN parameters (and thus over functions), of which we are taking N_{ens} samples. A substantial body of research characterises this distribution and related ones as approximate Bayesian inference [99, 100, 101, 102, 103]. While we do not make use of this Bayesian perspective in our work, we need to keep in mind that every aspect of the training process influences the resulting posterior distribution and must be considered a type of inductive bias.

In the rest of this report, we will refer to a generic parametric approximation $V^\Theta = (\mu^\Theta, \sigma^\Theta)$ of the value function, consisting of a mean and uncertainty

2. FUNDAMENTALS

estimate. In our experiments, they are defined by taking the mean and standard deviation of the NN ensemble predictions:

$$\mu^{\Theta}(x) = \frac{1}{N_{\text{ens}}} \sum_{i=1}^{N_{\text{ens}}} V_{\theta_i}(x) \quad (2.12)$$

$$\sigma^{\Theta}(x) = \sqrt{\frac{1}{N_{\text{ens}}} \sum_{i=1}^{N_{\text{ens}}} (V_{\theta_i}(x) - \mu^{\Theta}(x))^2} \quad (2.13)$$

where $\Theta = (\theta_1, \dots, \theta_{N_{\text{ens}}})$ contains the parameters θ_i of each NN ensemble member, and $V_{\theta}(x)$ denotes the output of an individual NN with parameters θ and input x . However, we point out that none of our methods and results make use of the specific structure of the parametric approximation V_{Θ} . Thus it can be replaced by any other suitable method for parametric function approximation with uncertainty estimates.

Proposed Methods

3.1 Level-Set Dynamic Programming

We propose *Level-Set dynamic programming* as a conceptual approach to solving the optimal control problem (2.2). The core idea is to learn the value function from locally optimal trajectories in ascending order of cost-to-go.

This means that at any $x \in \mathcal{X}$, the globally optimal solution, which has the lowest cost-to-go, is learned first. Suboptimal solutions are only considered later, allowing us to detect and discard them. This concept is illustrated in Figs. 3.1 and 3.2.

If *all* locally optimal trajectories at every point are available and we “learn” precisely from infinite data, this is a trivial restatement of the fact that the global solution is the best among all locally optimal solutions (2.1.5). Beyond this, we consider the level-set DP approach promising for practical application for the following reasons:

- It avoids ever learning suboptimal solutions – the optimal one is always learned first. This has obvious efficiency benefits over alternatives which may first learn a suboptimal solution, only to later discard it in favour of the optimal one.
- It avoids explicit comparisons between different local solutions to find the best one. Rather, each local solution is only compared with the learned value function. This is well suited to the practical case of training data which describes distinct local solutions in overlapping regions, but not at precisely the same points.

Still, we stress that there is no guarantee that this works in the practical setting, with finite data and NN approximation of V . Rather, this is what we aim to find out empirically.

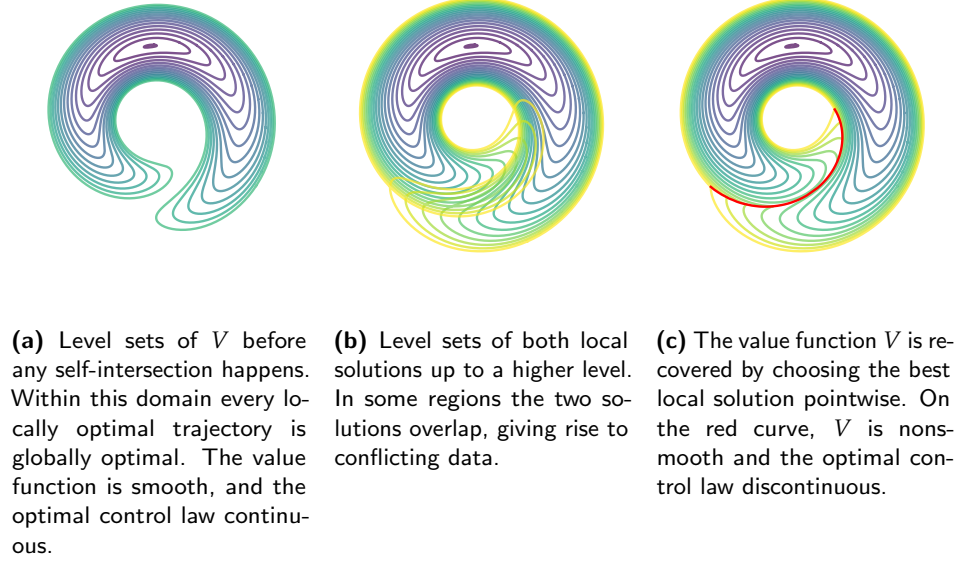


Figure 3.1: Illustration of value level sets in the case of distinct local solutions.

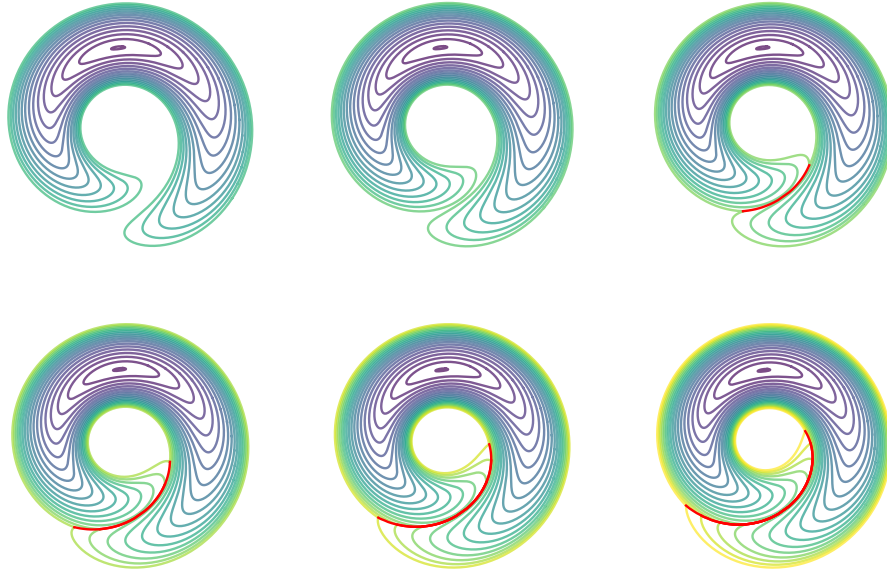
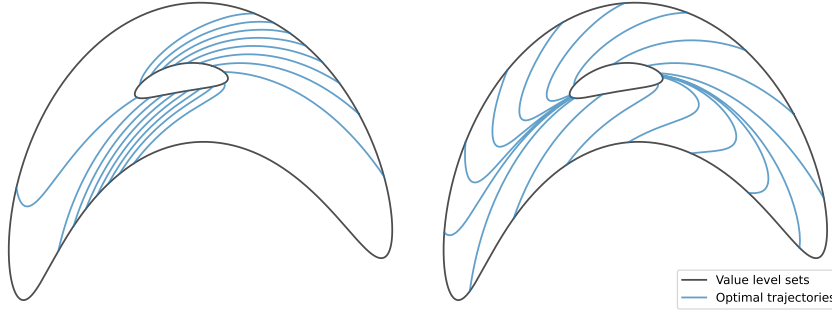


Figure 3.2: If local solutions are added in ascending order of cost-to-go, then at any particular point the global solution (i.e. the local solution with lowest cost-to-go) is added first. In this way, V is equivalently characterised as the cost-to-go of the *first* local solution to be added at any particular point.



(a) Optimal trajectories which are started at arbitrary points on the lower value level set will in general not sufficiently cover the next sublevel set.

(b) Ideally, we would want to cover all regions of the next sublevel set with optimal trajectories. However, that is unachievable with PMP backward integration, as we do not know the required boundary conditions in advance.

Figure 3.3: Connecting a value level set to the next one with a set of optimal trajectories.

3.2 Active Learning Formulation

3.2.1 Main Idea

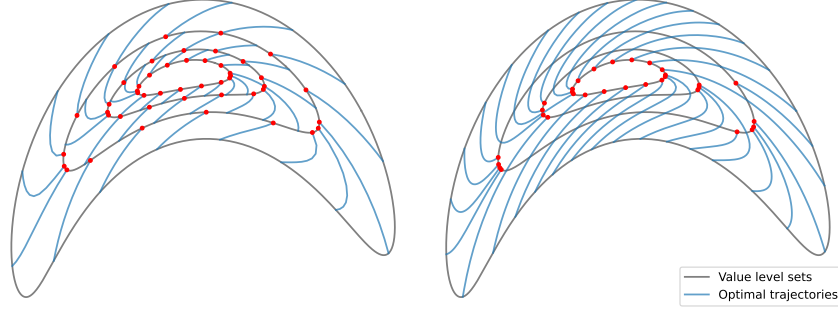
We have introduced the main idea of level-set DP under the assumption of *all* local solutions being available at every x . Of course, this is not achievable practically – instead, we generate a finite set of optimal trajectories, and approximate V based on that data.

We illustrate the central tradeoffs of data generation in Figs. 3.3 and 3.4, and attempt to practically reach the situation in Fig. 3.4b with our methods.

We implement level-set DP practically with the following high-level steps:

- Find the largest sublevel set in which V is known with sufficiently low uncertainty
- Find a set of locally optimal trajectories arriving at the known sublevel set by solving (2.7)
- Use those trajectories to improve the approximation of V on a larger sublevel set

These steps are iterated until the known sublevel set entirely contains the domain of interest. In the following section, we make this intuitive idea more concrete and present our main algorithm.



(a) Optimal trajectories which are only integrated from one level set to the subsequent one, where a completely new set of optimal trajectories is initialised based on the approximate value function.

(b) Optimal trajectories which are integrated from one level set beyond the subsequent one. This means most of the data required for learning the next level set is already available. New trajectories only have to be initialised in regions not covered by old ones.

Figure 3.4: Two different approaches of generating new trajectories to learn the next level set. Every re-initialisation of an optimal trajectory (which means incurring error due to function approximation) is highlighted in red.

3.2.2 Algorithm

Consider a parametric value function approximation $V^\Theta = (\mu^\Theta, \sigma^\Theta)$ containing estimated mean μ^Θ and uncertainty σ^Θ , both mapping from $\mathcal{X}$ to $\mathbb{R}$. Our goal is to achieve an approximation with sufficiently low uncertainty. To make this goal concrete, we introduce an uncertainty threshold function $\sigma_{\max} : \mathcal{X} \rightarrow (0, \infty)$. With it, we define:

Definition 3.1 $x \in \mathcal{X}$ is a known point if $\sigma^\Theta(x) \leq \sigma_{\max}(x)$.

Definition 3.2 The known value level v_k (at iteration k) is the largest v for which the sublevel set $\mathcal{V}^\Theta = \{x \in \mathcal{X} : \mu^\Theta(x) \leq v\}$ contains only known points. We refer to the associated $\mathcal{V}_k^\Theta$ as the known sublevel set.

With these definitions, we propose Algorithm 1 as a practical implementation of Level-Set DP using active learning concepts. We present the core aspects of each component, while referring to Appendix A for further details.

Value Level Estimation

The function `EstimateValueLevel` (Line 2) finds the known value level v_k , as defined in Def. 3.2. Equivalently, we can search for the lowest $\mu^\Theta(x)$ among points which are *not* known:

$$v_k = \min_{x \in \mathcal{X}} \mu^\Theta(x) \quad \text{s.t.:} \quad \sigma^\Theta(x) > \sigma_{\max}(x) \quad (3.1)$$

Algorithm 1 Active Learning formulation of Level-Set DP**Require:** Initial approximate value function $V^\Theta = (\mu_0^\Theta, \sigma_0^\Theta)$ **Require:** initial dataset $\mathcal{D}$

```

1: for  $k \in \mathbb{N}$  do
2:    $v_k \leftarrow \text{EstimateValueLevel}(V_k^\Theta)$ 
3:   if  $v_k > V_{\max}$  then Return  $V_k^\Theta$ 
4:    $\hat{v}_{k+1} \leftarrow \text{SetValueTarget}(V_k^\Theta, v_k)$ 
5:    $\hat{\mathcal{X}} \leftarrow \text{ProposePoints}(\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1})$ 
6:    $\mathcal{D} \leftarrow \mathcal{D} \cup \text{GenerateTrajectories}(\hat{\mathcal{X}}, V_k^\Theta, v_k)$ 
7:    $\mathcal{D}, V_{k+1}^\Theta \leftarrow \text{TrainAndPrune}(\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1})$ 
8: end for

```

Next Value Target

The function SetValueTarget (Line 4) determines the value level $\hat{v}_{k+1}$ of the *next* sublevel set in which we aim to learn the value function. We fix a time horizon T_{target} as an upper bound of the time duration of trajectories going from $\partial\mathcal{V}_{k+1}$ to $\partial\mathcal{V}_k$. We then set $\hat{v}_{k+1}$ by choosing the largest value step $\hat{v}_{k+1} - v_k$ that ensures satisfaction of the chosen upper bound.

Proposals

The *proposals* selected in (Line 5) are points $\hat{x} \in \mathcal{X}$ where we want to obtain additional training data. In active learning, these points are typically chosen by maximising an acquisition function, which quantifies the usefulness of obtaining training data at any given point.

In line with this practice, we define an acquisition function which reflects our goal of obtaining training data at points where uncertainty is high compared to $\sigma_{\max}(x)$, but only within the next value sublevel set.

$$f_{\text{acq}}(x) := \frac{\sigma_k^\Theta(x)}{\sigma_{\max}(x)} \cdot \begin{cases} 1 & \mu_k^\Theta(x) \in [v_k, \hat{v}_{k+1}] \\ 0 & \text{otherwise} \end{cases} \quad (3.2)$$

We use the relative uncertainty $\sigma^\Theta(x)/\sigma_{\max}(x)$, rather than the uncertainty $\sigma^\Theta(x)$. This has a natural interpretation as the ratio by which the uncertainty needs to be decreased until x is considered known (Def. 3.1).

Batch Selection Because updating the approximation V_k^Θ is computationally expensive, we generate N_{batch} proposals, rather than just one.

This is an instance of batched active learning. To ensure diversity within the batch of proposed points, we mimic non-batched active learning by using a kernel-based proxy model of the reduction in uncertainty σ^Θ after adding

new data. The proxy is much cheaper to update than V_k^Θ , which is updated only after selecting the entire batch and generating training data.

The same model also allows consideration of data we already have from trajectories started at lower level sets, so that we only generate new trajectories in regions not yet covered by existing ones. This is also illustrated in Fig. 3.4b. For precise specification of the batch selection and other implementation aspects, we refer to Appendix A.4.

Trajectory Generation

For each proposed point $\hat{x} \in \hat{\mathcal{X}}$, we generate a locally optimal trajectory (Line 6). Instead of requiring it to start precisely at $\hat{x}$, we require only a trajectory which passes *close* to $\hat{x}$.

This means we can find it using PMP backward integration (2.1.4) rather than full trajectory optimization. As a heuristic for selecting a state $x_f \in \mathcal{V}_k^\Theta$ to start backward integration, we first find an *approximately* optimal trajectory from $\hat{x}$ to a point in $\mathcal{V}_k^\Theta$ in forward time. From there, we find the actual optimal trajectory in backward time. More precisely:

1. Solve (2.1) with control input $u_\Theta^*(x) = u^*(x, \nabla_x \mu_k^\Theta(x))$ starting at $\hat{x}$ in forward time, until reaching a state in the known sublevel set $x_f \in \mathcal{V}_k^\Theta$.
2. Solve (2.7) from boundary conditions $(x_f, \nabla_x \mu^\Theta(x_f), \mu^\Theta(x_f))$ in backward time, until reaching some upper threshold $v_{\text{upper}} > \hat{v}_{k+1}$.

Training and Pruning

Among the generated optimal trajectories, there may be distinct local solutions, leading to conflicting training data. We address these conflicts with the following three components:

- A curriculum learning scheme, ensuring that at each x the optimal solution is learned before any suboptimal ones
- A Huber-like loss function that is robust against conflicting points, and which gives less weight to data that corresponds to a worse solution than the one being represented by V^Θ
- Removal of the data points deemed suboptimal or conflicting, and retraining to remove their influence on V^Θ

We briefly explain each component here – further details can be found in A.6.

Curriculum Learning In section 3.1, we established that if points of locally optimal trajectories are added to the training data set in order of increasing cost-to-go, the global optimum is learned first at each point, with points from suboptimal trajectories only being added later.

However, this property is lost if the cost level is increased in discrete steps: Among all local solutions with cost-to-go in the interval $[v_k, \hat{v}_{k+1}]$, there may still be distinct local solutions at the same x . To avoid this, we increase the value level continuously from v_k to $\hat{v}_{k+1}$ during the training run.

Loss function The specific form of curriculum learning ensures that at each point, the optimal solution is introduced to training data before any suboptimal ones. However, this alone does not lead to V_k^Θ being a useful smooth approximation of V within $\mathcal{V}_k^\Theta$. Practically, we need to handle the following two difficulties:

- i) When suboptimal solutions are added to training data after the optimal one is already learned, V_k^Θ should continue representing the optimal solution, without being significantly influenced by the suboptimal one.
- ii) Necessarily, there will be a region close to the nonsmooth points of V where the V_k^Θ interpolates between adjacent solutions, rather than approximating any one of them accurately. This leads to a higher loss than in smooth regions – this high loss should not hinder the overall training process.

We attempt to address these situations practically by choosing a suitable loss function. A precise definition of the loss function is given in Appendix A.6, along with further design and implementation considerations.

Conflict removal After the curriculum learning run, there is typically a small fraction of data points which V_k^Θ does not accurately fit. We assume that these points either belong to a suboptimal solution, or that they are so close to a nonsmooth point of V that they are of no practical use.

We discard those points and update V^Θ again with only the remaining data, allowing any artifacts from the removed data to be removed. More precise notes about this removal are found in A.6.

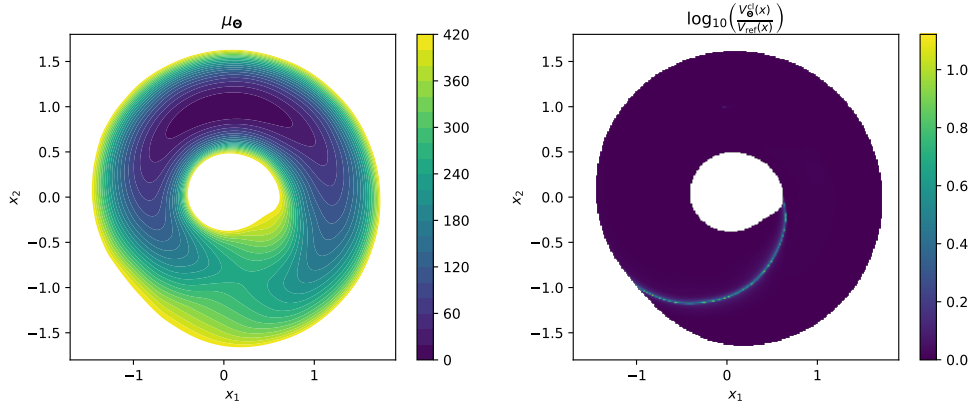
Experiment Results

4.1 Orbits System

We apply our proposed methods to the two-dimensional “Orbits” problem, defined in Appendix B.1. An approximate value function is computed up to the value level of $V_{\max} = 420$. The algorithm reaches this level after 23 iterations with the parameters given in Appendix C.

We compare the closed-loop cost $V_{\Theta}^{\text{cl}}(x)$ achieved by the resulting control law (Appendix D), and compare it with the reference solution $V(x)$ (Appendix E). The results are visualised in Fig. 4.1 and 4.2.

Additionally, we plot the learned value function, closed-loop cost, and ref-



(a) Learned mean value μ_{Θ} .

(b) Closed-loop cost $V_{\Theta}^{\text{cl}}(x)$ compared to reference solution $V_{\text{ref}}(x)$.

Figure 4.1: Learned value function, and closed-loop control cost compared to reference solution. Both are visualised for x with $\mu_{\Theta}(x) \leq V_{\max}$.

4. EXPERIMENT RESULTS

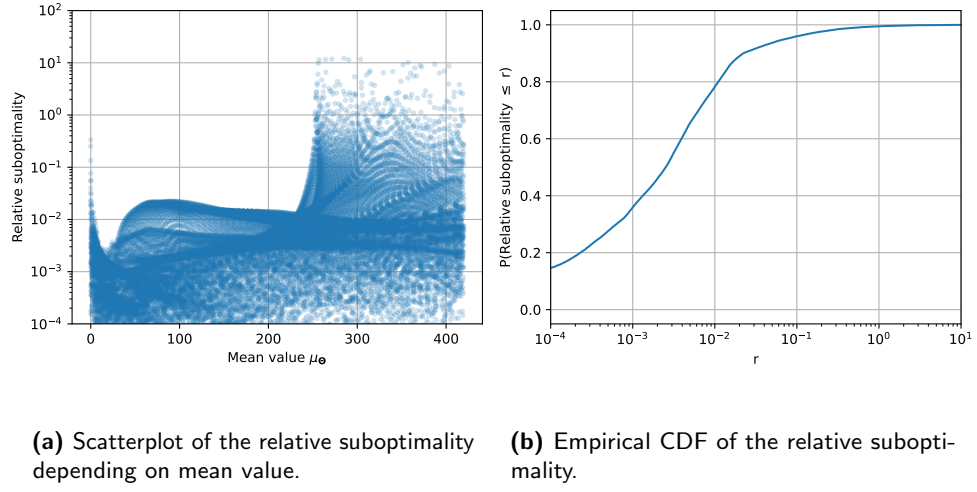


Figure 4.2: More visualisations of the *relative suboptimality* $\frac{V_{\Theta}^{\text{cl}}(x)}{V_{\text{ref}}(x)} - 1$. The relative suboptimality is evaluated on a uniform dense grid covering $[-2, 2]^2$ with 256 points in each direction, but only the points with $\mu_{\Theta}(x) \leq V_{\text{max}}$ are visualised.

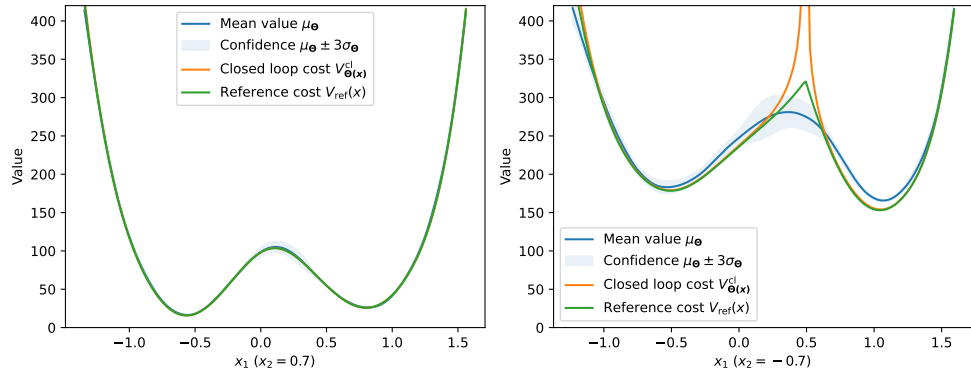


Figure 4.3: Learned value function, closed loop control cost, and reference cost along two lines through state space.

erence cost along two specific lines through state space in Fig. 4.3, one of which passes through a nonsmooth point of V .

4.2 Flatquad System

We address the more challenging task of optimally controlling a two-dimensional version of a quadrotor, which has a six-dimensional state space. The full problem definition is given in Appendix B.2.

We calculate an approximate value function up to a value level of $V_{\max} = 2000$. The algorithm reaches this level after 18 iterations with the parameters given in Appendix C.

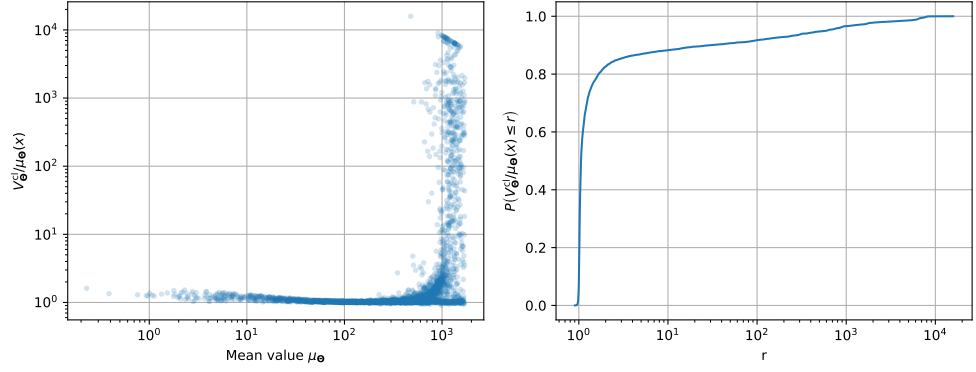
To assess the result, we evaluate the *relative cost* $\frac{V_{\Theta}^{\text{cl}}(x)}{\mu^{\Theta}(x)}$ at a large number of x with $\mu^{\Theta}(x) \leq V_{\max}$, which are sampled as described in A.2. $V_{\Theta}^{\text{cl}}(x)$ is obtained as described in D. This data is visualised in Fig. 4.4.

Additionally, we evaluate and plot the following quantities along six different curves $\gamma_i : [0, 1] \rightarrow \mathcal{X}$:

- The learned mean value $\mu^{\Theta}(x)$, with 3σ confidence bands
- The closed loop cost $V_{\Theta}^{\text{cl}}(x)$
- Globally and locally optimal reference costs (Details in E)

This data is visualised in Figs. 4.5 and 4.6.

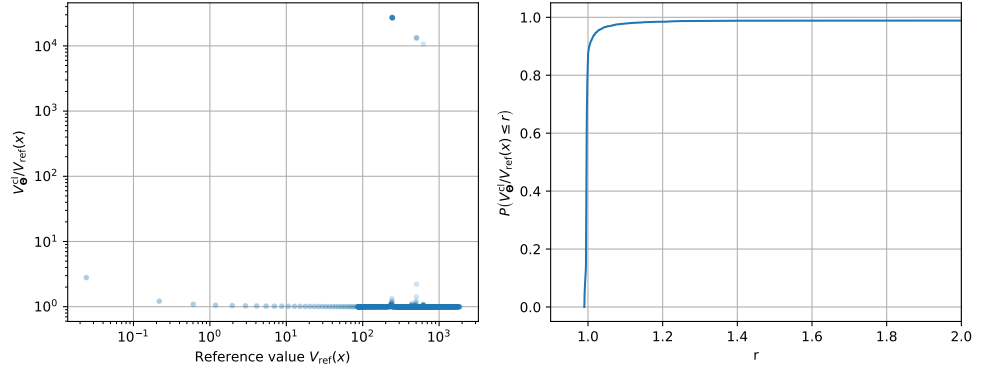
4. EXPERIMENT RESULTS



(a) Scatterplot of the relative cost depending on mean value

(b) Empirical CDF of the relative cost

Figure 4.4: Visualisations of the *relative cost* $\frac{V_{\Theta}^{\text{cl}}(x)}{\mu_{\Theta}(x)}$ with respect to the learned mean value μ^{Θ} , evaluated at x sampled according to A.2



(a) Scatterplot of the relative cost depending on reference value

(b) Empirical CDF of the relative cost

Figure 4.5: Visualisations of the *relative cost* $\frac{V_{\Theta}^{\text{cl}}(x)}{V_{\text{ref}}(x)}$ with respect to reference value $V_{\text{ref}}(x)$, evaluated at x on curves γ_i

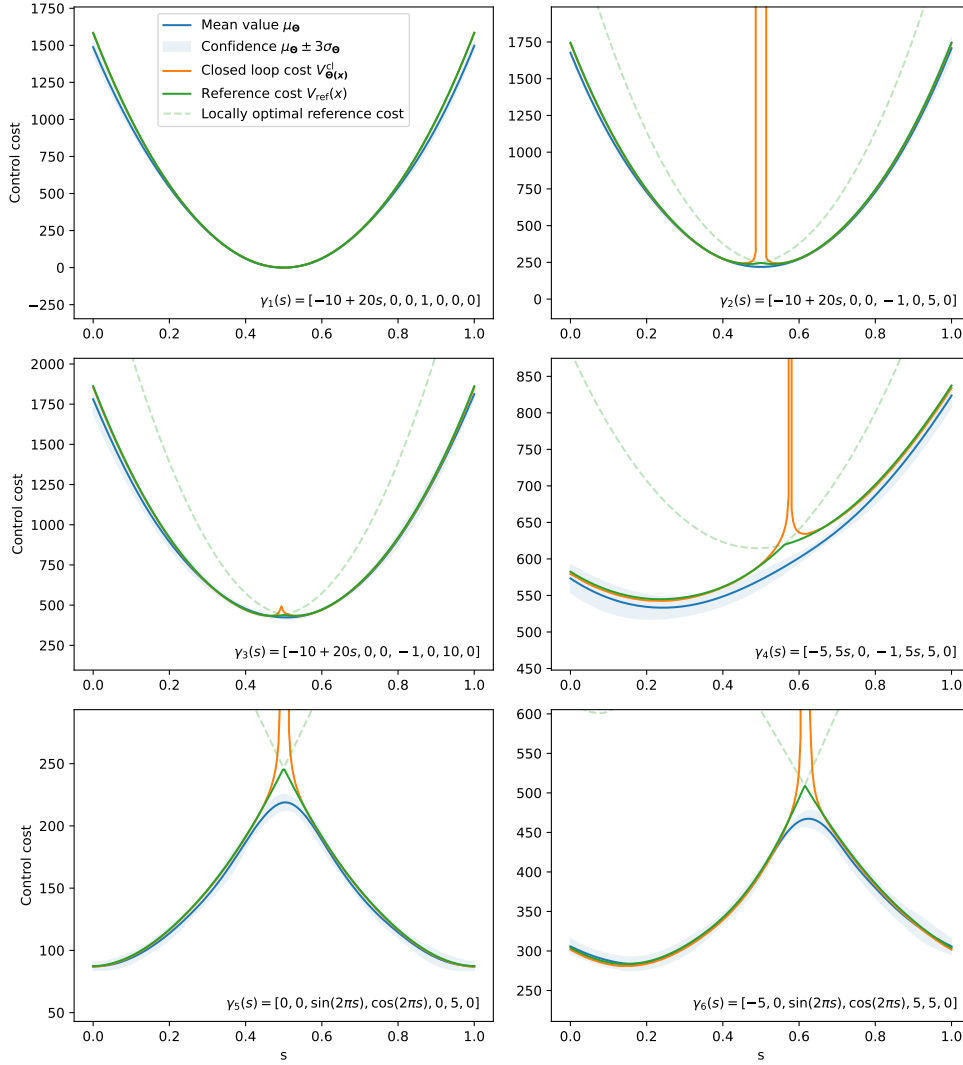


Figure 4.6: Control costs along initial state sweeps, given by curves $\gamma_i : [0, 1] \rightarrow \mathcal{X}$. We plot the learned value function mean μ_θ and 3σ confidence band, the closed loop control cost, and the reference cost given by a trajectory optimiser as described in Appendix E.

Discussion

5.1 Discussion of Results

Core functionality In our practical experiments, we achieve the goal stated in Section 1.2: The algorithm finds a smooth approximation of the solution V within the specified domain. In regions where V is smooth, the closed-loop cost is competitive with the reference solution V_{ref} given by trajectory optimisation.

This is despite applying several heuristics and approximations. In particular, the level-set idea introduced in 3.1 is not guaranteed to work in the practical setting with finite data and NN approximation. Also, our method incurs approximation errors whenever a trajectory is initialised with information from V^Θ , and those errors can accumulate with each iteration.

While we can make no claims that the presented methods work for more challenging problems, the results validate our methods for two simple problems and show that the resulting control law can be sufficiently close to optimality for practical application.

Smooth approximation of nonsmooth V While our solution is accurate for most initial states, its accuracy deteriorates close to nonsmooth points of V . Typically, there is a region around nonsmooth points where our control cost is significantly worse than the reference solution. In this region, our solution represents an interpolation between the adjacent solutions, rather than any one of them. For some initial states, this causes the controller to fail in stabilising the system at all.

We approximate the discontinuous optimal control law with a continuous function u_Θ^* ¹. In this setting, is impossible to achieve arbitrarily low approximation error on both sides of a discontinuity simultaneously. Moreover, a

¹Because our value approximation V_k^Θ is smooth, its gradient $\lambda = \nabla_x V_k^\Theta$ is also smooth,

wide class of dynamical systems cannot be globally stabilised by any continuous policy, even without considering optimality [104, 105, 27, 106]. Therefore, the observed limitation cannot be avoided within the paradigm of continuous policy approximation.

Unfortunately, it also seems practically challenging to alleviate the limitation. In our second experiment (Section 4.2) the volume of initial states for which our policy fails is significant, with 12% of the sampled initial conditions x having $V_{\Theta}^{\text{cl}}(x) > 10\mu^{\Theta}(x)$. Even arriving at this result was challenging, requiring various implementation tricks and extensive hyperparameter search. While we may try to further decrease the fraction of states where the smooth approximation is problematic, the only reliable way we have found to do so is to generate more data at each iteration (See Appendix F).

We use the NN and its training process not only as a tool for function approximation, but also as an implicit smoothness assumption specifying “how smooth” our smooth approximation of V should be. This happens through our mechanism of discarding data which violates the implicit smoothness assumption (i.e. which is not accurately approximated by the NN – see 3.2.2). From this perspective, it is unsurprising that the precise properties of the final approximation are practically not straightforward to tune.

Active learning We have taken inspiration from active learning for our methods. Usually, active learning is most useful in situations where labelling data is expensive and we want to be *sample efficient*, i.e. find a good approximation of the function of interest with only a limited number of training samples.

We have identified the approximation of V close to nonsmooth points as a central limitation of our approach, which in our experiments is only practically alleviated by generating large numbers of optimal trajectories. This overshadows any attempt at making the overall algorithm more sample efficient.

Due to this difficulty, we were not able to conduct thorough experiments comparing different active learning ingredients such as acquisition function or batching implementation. At this point, we cannot conclude as to whether our version of uncertainty sampling presents a practical advantage over simpler alternatives like uniform sampling.

5.2 Comparison with Existing Research

Trajectory optimisation Compared to TO, our method removes the guesswork of finding an initial guess for a local optimiser, and is able to approxi-

and in particular continuous. Due to continuity of Eq. 2.3 in both arguments x and λ , the approximate optimal control policy u_{Θ}^* is also continuous.

mate the global solution. Moreover, the solution can be retrieved in real-time by evaluating a neural network and solving a small convex optimisation problem (2.3), which is preferable to the relatively large, nonconvex problems arising from TO.

However, TO can efficiently find local solutions of many practical, high-dimensional problems. Our approach is based on dynamic programming, and can be expected to suffer from the curse of dimensionality [107], as well as additional practical challenges when applying it to high-dimensional problems.

TO also avoids the limitations we observe close to nonsmooth points of V – while not necessarily finding the globally optimal solution, it can find an accurate approximation of one particular local solution, regardless of different local solutions which may exist.

Learned trajectory optimisation Compared to many previous works about learning locally optimal solutions from TO, we show a method for finding distinct local solutions and constructing a smooth approximation of the globally optimal control law based on them.

There are several alternative, more advanced methods of function approximation which are capable of representing discontinuity. Among them are implicit or energy-based models [31], diffusion models [32, 33], or mixture-of-experts models [25, 27]. We demonstrate that a smooth approximation of the globally optimal control law can also be represented by comparatively simple neural networks. However, our smooth approximation fails to reproduce the behaviour of the optimal policy close to nonsmooth points.

Discrete-time DP The continuous-time optimal control problem can be approximately solved by discretising it in time, and solving it with discrete-time methods of approximate dynamic programming based on the Bellman Equation.

In contrast to that approach, our method completely isolates time discretisation from the rest of the algorithm, enabling the use of any suitable ODE solver. Particularly, there is no explicit input discretisation: Whereas discrete-time methods generally rely on zero-order hold input discretisation (which is known to converge only linearly to the continuous time solution with decreasing timestep), we enable the use of higher order ODE solvers, benefitting from higher-order convergence to the true solution.

However, unlike our approach, discrete-time methods are capable of representing discontinuous policies based on a smooth approximate value function. Generally, discrete-time methods are more popular and well-researched, and

have been shown to practically address significantly more challenging problems than the ones in our experiments.

Continuous-time DP In contrast to methods described in 1.3.3, we work with two separate components of data generation and function approximation. While this does not generally present an advantage or disadvantage, we argue that this separation enables more freedom in selecting the individual algorithmic components, at the expense of making the overall implementation more complex.

The publications in our literature review all work with smooth value function approximations, therefore they face very similar limitations as our approach in that regard, as discussed in 5.1.

PINNs have only been shown to solve two-dimensional optimal control problems, whereas we address a six-dimensional system with relative success.

Methods based on the dual formulation of optimal control such as Sums-of-Squares generally require the problem data to be represented as polynomials. In contrast, we only require differentiability and the ability to simulate the system numerically, allowing for a wider class of problems to be addressed.

PMP backward integration Compared to previous experiments with PMP backward integration [63, 64, 65, 66], we introduce a NN approximation of the value function and show empirically that our particular approach scales to a simple six-dimensional problem.

5.3 Further Research

Nonsmooth value representation Our methods are unable to accurately represent V close to nonsmooth points. Future works can address this limitation in several ways:

- Learn a family of smooth functions, each one of which represents V on a subset of the state space. This is the core idea behind Mixture-of-Experts models, and similar ideas have been explored in an optimal control context [25, 26].
- Learn a distribution over values, costates, and/or inputs for each x . Such models have shown practical success in representing multimodality and distinct local solutions in the discrete time setting [32, 33], but typically require more computation at inference time.
- Learn some representation of the stable manifold $W^s \subset \mathcal{X} \times \mathbb{R}^p$ as defined in 2.1.5, or only the parts of it corresponding to globally optimal solutions. Conceptually, this is similar to the previous idea, but may be

better suited to the deterministic setting. We expect similar challenges of computationally heavy inference: To find the optimal input at some x , we have to (approximately) find all λ such that $(x, \lambda) \in W^s$.

- Continue with a smooth approximation of V , but find a way to recover the discontinuous control law from it. Discrete-time methods can already do this². Similarly, we could use our smooth value approximation as terminal cost of a short-horizon MPC solver. While this would certainly work, it would necessitate nonconvex optimisation at inference time, negating one key advantage of our approach and working in continuous time generally.

These ideas all amount to searching for better *inductive biases*, that is, modelling approaches better suited to the piecewise smooth nature of V .

Our results have relied on using relatively large neural networks and substantial training data, primarily due to the challenge of approximating V at nonsmooth points.

On the other hand, approximating V in smooth regions was also possible with less data and fewer NN parameters. By further developing these ideas, we might therefore hope to not only represent nonsmooth V but to do so with fewer data and smaller networks. However, definitive conclusions can only be reached by continued research.

Scalability We have addressed a 6D problem, and while nontrivial, this is still relatively small compared to many problems of practical interest. While we may attempt to solve larger problems with our current approach, we consider the development of a more suitable approach to function approximation, as outlined in the previous paragraph, to be a prerequisite for doing this practically.

Even then, our methods may or may not be suited to higher dimensional problems. For example, one central idea behind our method is that longer trajectories are preferable, as illustrated in Fig. 3.4b. However, in the higher dimensional case these longer trajectories from lower sublevel sets might occupy only such a small fraction of the volume of higher sublevel sets as to be practically useless.

If so, possibly a different approach somewhere between trajectory optimization and dynamic programming offers a more practical solution – Further research is needed to draw definitive conclusions.

²Because the greedy controller in discrete time is defined by minimising the one-step lookahead cost [108], it considers the (approximated) value of all states reachable within one timestep, rather than the value gradient at the current state. This in turn enables it to consider distinct local solutions and choose the best one.

Active learning components As outlined in 5.1, we did not thoroughly investigate many different choices in active learning components due to the difficulty of approximating nonsmooth V . However, if those difficulties are circumvented with one of the ideas from above, the door towards improving active learning components is opened, and we list some possible directions here.

Currently, the uncertainty σ^Θ is not representative of our actual goal of achieving low uncertainty in terms of V and $\nabla_x V$ simultaneously. A more appropriate uncertainty measure could take V and $\nabla_x V$ into account – we had to avoid this because in our implementation, $\nabla_x V$ always had very high variance close to nonsmooth points.

Moreover, there is no reason to expect that our kernel-based proxy of the uncertainty decrease when adding new training data (See 3.2.2 and A.4) reflects the actual model V^Θ accurately. [89] gives several more advanced alternatives, including ones based on the neural tangent kernel, which would match the actual NN much more closely.

Wider problem class We address the deterministic, continuous-time, infinite-horizon optimal control problem. This is relatively limited, and many practical problems include additional difficulties, such as state constraints or imprecise dynamics models.

The PMP becomes much more involved when adding state constraints or other types of nonsmooth problem data, and the optimal system in reversed time is not described by an ODE like (2.7), but a differential inclusion [109, 110]. It seems more promising to use existing, advanced trajectory optimisers, which can handle state constraints, rather than extend our approach to such problems. However, our paradigm of learning in level sets can still be used even when generating data with more advanced trajectory optimisers.

Imprecise dynamics models are usually addressed by modelling the imprecision as stochasticity, which is the dominant paradigm in RL. However, an alternative consists of modelling a *robust* control problem as a differential game.

In this setting, the goal is minimisation of the control cost while an adversarial disturbance tries to maximise it. The associated value function is given by the Hamilton-Jacobi-Isaacs (HJI) equation [111], and the associated minimum principle is very similar to (2.7), with the addition of an adversarial disturbance. One possible direction of further research is the development of practical methods to globally solve *robust* nonlinear control problems in a way similar to ours.

Discontinuous value function We have assumed continuity of V (Assumption 5) to isolate the difficulty of nonsmooth V . However, practical problems can also have discontinuous value functions, as seen by example in [24]. Further research could generalise our methods to handle this case in one of two ways:

- Find a way to precisely represent the discontinuity, with ideas similar to the ones given above to address nonsmooth V
- Explore approaches of finding practical, continuous approximations of discontinuous V

Larger input spaces While we have assumed $\mathcal{U}$ to be a compact, convex polytope (Assumptions 4 and 6), our method for practically solving (2.3) is a simple version of explicit convex optimisation (A.9), which has exponential complexity in the number of inequality constraints.

To address $\mathcal{U}$ with more constraints, existing and more advanced tools of explicit convex optimisation [112, 113] can be used to solve (2.3) in advance for all (x, λ) . These tools generally also have exponential complexity but can offer practical advantages. Specifically, they can eliminate active sets that are either infeasible or never optimal in advance.

Any iterative convex optimiser could be used within the ODE integrator, possibly with warmstarting between subsequent right-hand side evaluations. However, this introduces an additional source of approximation error, and can also be relatively expensive computationally, even if favourable to explicit convex optimisation.

More advanced approaches to solving similar problems exist [114, 115], which may be preferable to the simpler options listed above.

Theoretical analysis While we empirically validate our methods with two simple problems, many further questions remain unanswered.

Our method incurs approximation error whenever information given by V^Θ is used to initialise new locally optimal trajectories. While we attempt to practically limit this error (by only using information from V^Θ where the estimated uncertainty is low), we do not precisely analyse how this error evolves (in the worst case under suitable assumptions), which is an interesting topic for further research.

We also make a rather large “Leap of Faith” from the infinite-data setting to the realistic one with finite data and NN approximation. Further work can shrink this gap between theory and practice, and search for precise conditions under which similar methods work under realistic assumptions.

The purpose of such theoretical analysis might be questioned in light of the fundamental limitations in approximating the discontinuous control law with a continuous function. However, we have presented ideas on how this limitation could be addressed, and if subsequent works achieve practical success with those ideas, then a more thorough analysis is warranted.

5.4 Conclusion

We have presented an approach to approximate dynamic programming in continuous time. We iteratively construct a large dataset of locally optimal trajectories, which are found by simulating the system under optimal control inputs in backward time. We then extract an approximation of the value function, and thus the globally optimal control law from that dataset.

In contrast to finding a control law by online trajectory optimisation, we show a principled method of handling distinct local minima, and find an approximation which is efficiently evaluated for online use. In contrast to using existing methods of discrete-time methods of approximate dynamic programming as an approximation to solving the continuous-time problem, we enable the use of higher-order ODE solvers, requiring no time discretisation of the control input.

In practical experiments involving a 2D and a 6D problem, our methods compute accurate approximations of the optimal control law in most regions. However, we identify the points where the optimal control law is discontinuous as challenges: Our method is unable to accurately represent the discontinuity, and the resulting control law fails close to those points. We propose several ways to circumvent this limitation, as well as other directions for further research.

Appendix A

Implementation Aspects

A.1 LQR Initialisation

To initialise the entire algorithm, we solve the LQR problem of the linearised system, an approach also adopted by previous similar works such as [64, 65]. Subsequently, we use the LQR solution to obtain an initial set of optimal trajectories, and to find a first value approximation $\mathcal{V}^\Theta$.

LQR Problem We solve the canonical continuous-time, infinite horizon LQR problem in terms of $\Delta x = x - x_{\text{eq}}$ and $\Delta u = u - u_{\text{eq}}$:

$$V_{\text{LQR}}(\Delta x_0) = \min_{\Delta u} \int_0^\infty \Delta x(\tau)^\top Q \Delta x(\tau) + \Delta u(\tau)^\top R \Delta u(\tau) d\tau \quad (\text{A.1})$$

$$\text{s.t.: } \frac{d}{d\tau} \Delta x(\tau) = A \Delta x(\tau) + B \Delta u(\tau) \quad (\text{A.2})$$

$$\Delta x(0) = \Delta x_0 \quad (\text{A.3})$$

With $Q = \nabla_x^2 l(x_{\text{eq}}, u_{\text{eq}})$, $R = \nabla_u^2 l(x_{\text{eq}}, u_{\text{eq}})$, $A = \nabla_x f(x_{\text{eq}}, u_{\text{eq}})$ and $B = \nabla_u f(x_{\text{eq}}, u_{\text{eq}})$. The problem is well-posed thanks to Assumption 2. The solution for all Δx_0 is obtained by solving the continuous-time algebraic Ricatti equation [35, Eq. 6.28] and takes the form $V_{\text{LQR}}(\Delta x) = \Delta x^\top P_{\text{LQR}} \Delta x$. In case $\mathcal{X}$ is a submanifold of $\mathbb{R}^p$, we define and solve the LQR problem in the tangent space $T_{x_{\text{eq}}} \mathcal{X}$ as outlined in [116, Section 8.8.3].

Initialisation After solving the LQR problem, we have the approximate value function $V_{\text{LQR}}(x) = (x - x_{\text{eq}})^\top P_{\text{LQR}} (x - x_{\text{eq}})$, which is valid for x close to x_{eq} . We construct an initial dataset $\mathcal{D}$ containing locally optimal trajectories according to Algorithm 2.

Algorithm 2 Construction of Initial Dataset $\mathcal{D}$

```

Obtain LQR solution  $P_{\text{lqr}}$ 
 $L_{\text{LQR}} \leftarrow \text{CholeskyDecomposition}(P_{\text{LQR}})$ 
for  $k = 0, 1, \dots, N_{\text{batch}}$  do
  Sample  $z \sim N(0, I)$ 
   $x \leftarrow x_{\text{eq}} + \sqrt{V_f} L_{\text{LQR}}^{-1} z$ 
   $\mathcal{D}_k \leftarrow \text{Solve Eq. (2.7) starting at } (x, \nabla_x V_{\text{LQR}}(x), V_{\text{LQR}}(x))$ 
   $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_k$ 
end for

```

Notice that the state x is selected such that $V_{\text{LQR}}(x) = V_f$. Therefore, V_f determines the size of the initial sublevel set in which we accept the LQR approximation as true. We always set $V_f = 0.001$.

After generating the dataset $\mathcal{D}$, we find an initial parametric approximation V^Θ based on that data and proceed with Algorithm 1. In contrast to later iterations, we train with more epochs and with an exponential decay learning rate schedule – see A.6 and C.

A.2 Value Level Estimation

Uncertainty threshold In our experiments, we use an uncertainty threshold σ_{max} of the form

$$\sigma_{\text{max}}(x) := \sigma_{\text{abs}} + \sigma_{\text{rel}} \mu^\Theta(x) \quad (\text{A.4})$$

which is motivated by the fact that typically, the value function spans several orders of magnitude, and we want good relative accuracy at all scales.

Finite data approximation The known value level is estimated by solving the optimisation problem (3.1) not over all of $\mathcal{X}$, but only a finite subset $\mathcal{X}_{\text{test}} \subset \mathcal{X}$, which is an application of sampling-based optimisation. Note that due to approximating the problem with finitely many points, we will overestimate the known value level slightly.

The test points $\mathcal{X}_{\text{test}}$ are sampled from a problem-specific distribution. For low-dimensional problems, we sample from a uniform distribution over a large box, and we empirically ensure that it completely contains the part of the state space we are interested in.

For higher-dimensional problems, we encounter the problem that even with a large number of uniform samples, not enough of them are sufficiently close to the equilibrium x_{eq} to accurately determine the known sublevel set $\mathcal{V}_k^\Theta$, which is initially very small. To remedy this, we scale half of the test points down by a factor of 10^t , with $t \sim \mathcal{U}(-k, 0)$ and $k = 2$. With this practical

adaptation, we have enough points close to x_{eq} for accurate estimation of small value levels.

In case the $\mathcal{X}$ is a manifold, we apply the scaling procedure only to the entries of the state vector which do not affect the constraint function m . For the other entries, we instead sample from a uniform distribution over the entire manifold, which in our examples is relatively simple.

A.3 Next Value Target

The value step $v_{k+1} - v_k$ (Section 3.2.2) is related to the time duration of trajectories going from $\partial\mathcal{V}_{k+1}$ to $\partial\mathcal{V}_k$: for any solution (x, λ, v) of (2.7) with $v(t_0) = v_{k+1}$ and $v(t_1) = v_k$, we have the upper bound

$$v_k - v_{k+1} \leq (t_1 - t_0) \max_{x: v_k \leq V(x) \leq v_{k+1}} -l(x, u^*(x)) \quad (\text{A.5})$$

obtained by replacing the right-hand side of (2.7c) with its maximum over the region of interest and integrating. Therefore, by selecting v_{k+1} such that the above inequality is satisfied with equality, we can (conservatively) satisfy the desired upper bound T_{target} .

The upper bound (A.5) requires knowledge of the value function within the value band $\mathcal{V}_{k+1} \setminus \mathcal{V}_k$. However, we do not have this information yet. As a proxy, we instead evaluate $-l(x, u^*)$ at points from already known locally optimal trajectories which pass through $\partial\mathcal{V}_k$. Specifically, from each trajectory which has points with cost-to-go v both above and below v_k , we take the first point with $v \leq v_k$ and evaluate $l(x, u^*)$ there.

Not only is this a sampling-based approximation of the original optimization problem (A.5), but we are also optimising over a different domain: instead of $\mathcal{V}_{k+1} \setminus \mathcal{V}_k$, we consider only points within the sublevel set $\mathcal{V}_k$, but close to its boundary.

Therefore, we cannot guarantee satisfaction of the chosen upper bound T_{target} anymore. However, we observe that practically it often is satisfied, and even when it is violated slightly the overall algorithm still works as intended.

A.4 Proposals

Inspired by [89], we select a batch of proposals by imitating a non-batched active learning process, where the proposal is always a maximiser of the acquisition function.

Instead of updating V^Θ itself after each proposal, we update only a very simple, kernel-based proxy σ_p of the uncertainty σ^Θ . Before proposing any

points, we update the proxy uncertainty with already existing data points, so that the new proposals do not lead to redundant data.

The proxy uncertainty σ_p gives rise to a corresponding proxy acquisition function f_p . We then maximise f_p to arrive at the next proposal. The selection process is defined concretely in Algorithm 3.

Algorithm 3 Batch Proposal Selection

```

function PROPOSEPOINTS( $\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1}$ )
     $\hat{\mathcal{X}} \leftarrow \emptyset$ 
     $(\mu^\Theta, \sigma^\Theta) \leftarrow \mathcal{V}_k^\Theta$ 
     $\sigma_p \leftarrow \sigma^\Theta$ 

    ▷ Update proxy uncertainty based on existing data
    for  $(x, \lambda, v) \in \mathcal{D}$  do
        if  $v \in [v_k, \hat{v}_{k+1}]$  then
             $\sigma_p \leftarrow (x \mapsto \sigma_p(x)(1 - k(\hat{x}, x)))$ 
        end if
    end for

    ▷ Propose new points sequentially
    for  $i = 0, \dots, N_{\text{batch}}$  do
         $f_p \leftarrow (x \mapsto \sigma_p(x) / \sigma_{\max}(x))$       ▷ Define proxy acquisition function
         $\hat{x} \leftarrow \operatorname{argmax}_{x: \mu^\Theta(x) \in [v_k, \hat{v}_{k+1}]} f_p(x)$       ▷ Find proposal
         $\hat{\mathcal{X}} \leftarrow \hat{\mathcal{X}} \cup \{\hat{x}\}$ 
         $\sigma_p \leftarrow (x \mapsto \sigma_p(x)(1 - k(\hat{x}, x)))$       ▷ Update proxy uncertainty
    end for
    return  $\hat{\mathcal{X}}$ 

end function

```

The kernel $k : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ describes the fraction of uncertainty which is removed in the vicinity of a new data point. We use a scaled RBF kernel in our experiments:

$$k(x, x') = \alpha \cdot \exp \left(-\|D^{-1}(x - x')\|_2^2 \right) \quad (\text{A.6})$$

where D is a diagonal matrix defining the kernel length scale in each direction. We set $D = \eta \operatorname{diag}(r)$, where $r_i = \max_{x \in \mathcal{V}_k^\Theta} x_i - \min_{x \in \mathcal{V}_k^\Theta} x_i$ is the entry-wise extent of the current value sublevel set and $\eta < 1$ is a manually chosen scaling factor.

Algorithm 3 involves modifying the proxy uncertainty σ_p . This is a function from $\mathcal{X}$ to $[0, \infty)$, which we cannot represent exactly – to arrive at a practical

approximation, we carry out the entire algorithm not over the entire state space $\mathcal{X}$, but a finite subset $\mathcal{X}_p \subset \mathcal{X}$. Then, we can represent functions as a simple vector of function evaluations at each point $x_p \in \mathcal{X}_p$.

The subset $\mathcal{X}_p$ is obtained by rejection sampling: We sample uniformly from a box $\mathcal{B} = [x_1^l, x_1^u] \times \dots \times [x_n^l, x_n^u] \subset \mathcal{X}$ with $\mathcal{V}_{k+1}^\Theta \subset \mathcal{B}$, and then accept only the samples that lie in $\mathcal{V}_{k+1}^\Theta \setminus \mathcal{V}_k^\Theta$. We continue this procedure until we have 16384 points.

If $\mathcal{X}$ is a manifold, we adapt the sampling procedure for the relevant coordinates as described in A.2.

A.5 Trajectory Generation

As described in 3.2.2, we find a locally optimal trajectory for each proposal $\hat{x}$ by first finding an approximate optimal trajectory starting at $\hat{x}$ and arriving in $\mathcal{V}_k^\Theta$, and then finding a corresponding locally optimal trajectory in backward time by solving (2.7).

In practice, we make two small adaptations to the steps given in 3.2.2:

1. When finding the forward trajectory, we only stop when reaching a state x where $\mu^\Theta(x) \leq v_k$ and $\sigma^\Theta(x) \leq \sigma_{\max}$. While in theory, v_k is defined such that the first condition implies the second, in practice we will overestimate v_k slightly, due to the sampling-based approximation of the optimisation problem (3.1) introduced in A.2. Therefore, we check the uncertainty condition $\sigma^\Theta(x) \leq \sigma_{\max}(x)$ again to avoid initialising optimal trajectories with inaccurate information.
2. Instead of setting a fixed threshold v_{upper} at which backward integration of the optimal trajectory is stopped, we integrate the ODE reaching a fixed number of solver steps, in our implementation 127.

If the forward trajectory never reaches the known sublevel set $\mathcal{V}_k^\Theta$, we simply discard the corresponding backward trajectory. This happens regularly, given that the approximate optimal controller is defined using the extrapolated value function. However, in practice the fraction of discarded trajectories is mostly below 25%.

A.6 Training and Pruning

In 3.2.2, we outlined on a high level how the problem of conflicting local solutions is addressed. Here, we present additional details and implementations aspects that were previously skipped over.

Curriculum learning At every iteration, we sample a minibatch of training points uniformly (without replacement) from the subset $\mathcal{D}_{[v_k, \hat{v}_{k+1}]} := \{(x, \lambda, v)_j \in \mathcal{D} : v \leq (1-s)v_k + s\hat{v}_{k+1}\}$, where $s = k/N$, with k the current training step and $N = N_{\text{epochs}} \cdot |\mathcal{D}_{[v_k, \hat{v}_{k+1}]}|$ the total number of training steps¹.

The NN ensemble is warm-started with the parameters from the previous iteration, and the learning rate is kept constant during the whole curriculum learning run.

Loss function The design goals outlined in 3.2.2 are achieved with two different components.

First, we penalise the costate error with a Huber-like loss function. This enables accurate approximation of the solution in regions where there is no conflicting data, while avoiding overly large gradients at conflicting labels, which we do not want to precisely fit anyway.

Second, we scale the entire loss function down by a factor s whenever the value label $\hat{v}$ is above the predicted value $v = \mu^{\Theta}(x)$. Crucially, we disable gradient backpropagation through this scaling factor to avoid artificially encouraging underestimation.

This is realised concretely with the following loss functions l_0 and l_1 , which are used in Eq. 2.11:

$$l_0(v, \hat{v}) = w_v \cdot \left(e_v^{\text{rel}}(v, \hat{v})\right)^2 \cdot s(e_v^{\text{rel}}(v, \hat{v})) \quad (\text{A.7a})$$

$$l_1(\lambda, \hat{\lambda}) = w_{\lambda} \cdot h_{\delta} \left(e_{\lambda}^{\text{rel}}(\lambda, \hat{\lambda})\right) \cdot s(e_v^{\text{rel}}(v, \hat{v})) \quad (\text{A.7b})$$

depending on the relative value and costate errors:

$$e_v^{\text{rel}}(v, \hat{v}) := \frac{v - \hat{v}}{\varepsilon_v + \hat{v}} \quad (\text{A.8a})$$

$$e_{\lambda}^{\text{rel}}(\lambda, \hat{\lambda}) := \frac{\|\lambda - \hat{\lambda}\|_2}{\varepsilon_{\lambda} + \|\hat{\lambda}\|_2} \quad (\text{A.8b})$$

and the scaling factor

¹In contrast to standard implementations, where the whole dataset is shuffled in the beginning and then read sequentially, we have to access memory in random order, which can be slower than sequential reading. However, this effect is empirically not noticeable for the relatively small data sets in our experiments.

$$s(e_v^{\text{rel}}) := \min(1, \exp(\mu e_v^{\text{rel}})) \quad (\text{A.9})$$

where $h_\delta(z) := \delta^2 \left(\sqrt{1 + (z/\delta)^2} - 1 \right)$ is a smooth version of the classic Huber loss function, with an approximately quadratic region of $[-\delta, \delta]$ and derivative at infinity $\lim_{x \rightarrow \pm\infty} \frac{d}{dx} h_\delta(x) = \pm\delta$ [117].

Further tuning parameters are the positive weights w_v and w_λ with $w_v + w_\lambda = 1$, and small constants ε_v and ε_λ that weaken the loss terms additionally for very small $\hat{v}$ or $\|\hat{\lambda}\|_2$, respectively.

We use `jax.lax.stop_gradient` [118] to disable gradient propagation through the scaling factor s . This means that we are not technically optimising the given loss function anymore. Rather, we are following some vector field which may or may not be the gradient of some different loss function. However, in our experiments the disabling of backpropagation through s seems to achieve the stated objective.

Conflict removal The Huber loss function, and its smoothed version h_δ , are typically used in regression tasks to achieve a degree of robustness against outliers. In our application we use it to detect conflicting points. We assume that in the absence of conflicts, the loss will be low enough to fall in the quadratic region of h_δ , i.e. $e_\lambda^{\text{rel}}(\lambda, \hat{\lambda}) \leq \delta$. Conversely, we consider any point with $e_\lambda^{\text{rel}}(\lambda, \hat{\lambda}) > \delta$ a *conflicting point* and remove it from consideration. This includes both suboptimal solutions, and ones which are optimal but so close to a nonsmooth point that our smooth approximation fails to accurately fit the gradient label.

We also remove all points that precede any suboptimal or conflicting point on their respective trajectory. This is motivated by the fundamental dynamic programming principle [34, 108], which states that if a trajectory $x : [0, T] \rightarrow \mathcal{X}$ is optimal, then the same trajectory restricted to the subinterval $[t, T]$ with $t \in (0, T)$ is also optimal. Its contrapositive states that if a trajectory is suboptimal on $[t, T]$, then it must also be suboptimal on $[0, t]$, so we can remove the corresponding segment.

After removing those points, we update the approximation V^Θ , this time without curriculum learning, i.e. with the full training dataset. This allows any influence of the removed points to vanish from the learned function.

A.7 Solving ODEs on Manifolds

When numerically solving ODEs with some known invariant manifold, such as our state space $\mathcal{X} \subset \mathbb{R}^p$, the numerical solution will typically drift away from that invariant manifold. To keep it physically meaningful, we must

ensure this error stays reasonably small. In [119] this problem is discussed in more detail, along with several options of addressing it practically.

We do so by using a *projection method*: After each ODE solver step, the state vector is projected back onto the state space manifold with the minimum-distance projection $\Pi_{\mathcal{X}}(z) = \operatorname{argmin}_{x \in \mathcal{X}} \|x - z\|_2^2$.

This projection is available in closed form for a few simple manifolds. For example, it corresponds to dividing a vector by its 2-norm for $S^1 \subset \mathbb{R}^2$ or $S^3 \subset \mathbb{R}^4$, which represent embeddings of $\mathrm{SO}(2)$ and $\mathrm{SO}(3)$ respectively. For more challenging problems, this may not be the case. Alternatively, Newton's method together with the constraint function m can be used to define a similar operation – see [119] for more details.

To avoid potential numerical issues such as deteriorating ODE solver performance due to unnecessarily large entries in λ , we remove the (for our purposes meaningless) normal component by applying the projection $\Pi_{T_x^* \mathcal{M}}$ as defined in 2.1.6 after each ODE solver step.

This approach of using a general-purpose ODE solver with backprojection after each step is both simple and general. However, we point out that for each specific problem there are likely better choices for solving it numerically. For the simple systems we are considering, numerical simulation is a vanishingly small part of the overall computational cost, and therefore we are not very concerned about simulating as efficiently as possible.

A.8 Tangent and Cotangent Projections

In various places we require the projections $\Pi_{T_x \mathcal{X}}$ and $\Pi_{T_x^* \mathcal{X}}$ as defined in 2.1.6. Here, we outline how they are obtained numerically. We start with the projection onto the normal space $N_x \mathcal{X}$, which in general can be found as:

$$\Pi_{N_x \mathcal{X}} = BB^\dagger \tag{A.10}$$

where $B = [b_1, \dots, b_{p-n}] \in \mathbb{R}^{p \times (p-n)}$ is any basis of the normal space at some $x \in \mathcal{X}$, and $(\cdot)^\dagger$ denotes the Moore-Penrose pseudoinverse. One natural choice for such a basis is $B := (\nabla_x m(x))^\top$.

From there, we find the projection onto the orthogonal complement $T_x \mathcal{M}$ as $\Pi_{T_x \mathcal{M}} = I - \Pi_{T_x \mathcal{X}}$. The projection onto the respective dual space is the same: $\Pi_{T_x^* \mathcal{M}} = \Pi_{T_x^* \mathcal{X}}$.

If instead of a general basis B , we have an orthonormal basis $\tilde{B}$ of $N_x \mathcal{X}$, then the pseudoinverse simplifies to transposition, and we have:

$$\Pi_{N_x \mathcal{X}} = \tilde{B} \tilde{B}^\top \tag{A.11}$$

which is preferable to calculating the B^+ , if a suitable $\tilde{B}$ is easy to find.

In our practical examples, $\mathcal{X}$ is (at most) of codimension 1, making $\tilde{B}$ a simple column vector, which we find by normalising $(\nabla_x m(x))^\top$. If $\mathcal{X}$ has larger codimension, but can be written as the cartesian product of manifolds with codimension 1, then $(\nabla_x m(x))^\top$ is an orthogonal column basis of $N_x \mathcal{X}$, and can easily be made orthonormal by normalising each column.

We point out that if the normal space is of larger dimension than the tangent space, it may be more efficient to go the other way around: First, find a basis T of the tangent space, then find $\Pi_{T_x \mathcal{X}} = TT^\top$, and finally use that to find all the other relevant projections. This alternative approach requires the pseudoinverse of $T \in \mathbb{R}^{p \times n}$, rather than that of $B \in \mathbb{R}^{p \times (p-n)}$.

A.9 Hamiltonian Minimisation Problem

As outlined in 2.1.2, the optimal input u^* at each x is obtained by solving (2.3). Due to Assumption 6, this is a convex quadratic program and can be rewritten as:

$$u^*(x, \lambda) = \min_u \frac{1}{2}(u - u_{\text{eq}})^\top H_{uu}(u - u_{\text{eq}}) + H_u(u - u_{\text{eq}}) \quad (\text{A.12})$$

$$\text{s.t.: } Gu \leq h \quad (\text{A.13})$$

With $H_u = \partial_u H(x, u_{\text{eq}}, \lambda)$ and $H_{uu} = \partial_u^2 H(x, u_{\text{eq}}, \lambda)$, and $h \in \mathbb{R}^{n_c}$.

We solve it with a simple, “brute force” active set method [120]: Enumerate all active sets, find the solution for each one, and return the best solution that does not violate any constraints.

Algorithm 4 Active-Set convex QP solver

```

function FINDUSTAR( $x, \lambda$ )
  for  $I \in 2^{\{1, \dots, n_c\}}$  do
     $u_I \leftarrow$  Solve (A.12) with active set  $I$ 
     $c_I \leftarrow \frac{1}{2}u^\top H_{uu}u + H_u u + \begin{cases} 0 & Gu \leq h \\ \infty & \text{otherwise} \end{cases}$ 
  end for
   $I^* \leftarrow \text{argmin}_{I \in 2^{\{1, \dots, n_c\}}} c_I$ 
  return  $u_{I^*}$ 
end function

```

Here, 2^S is the *power set* of some set S , i.e. the set of all possible subsets of S . The problem (A.12) with a given active set $I \subseteq \{1, \dots, n_c\}$ denotes

the modified problem obtained by changing the inequality constraints I to equality constraints, and removing the rest. This modified problem is an equality-constrained QP, and we obtain the solution by solving the KKT system, which is a simple linear system [121, Section 10.1.1].

Typically, not all active sets I will result in a feasible problem. If the modified problem is infeasible, we also set $c_I = \infty$.

Enumerating all possible active sets quickly becomes infeasible for nontrivial problems, as we have $|I| = 2^{n_c}$. However, our problems have $n_c \leq 4$, making this unproblematic in our experiments.

Appendix B

Problem Definitions

B.1 Orbits

This is a two-dimensional toy problem to illustrate and validate our methods in a simple setting. With $\mathcal{X} = \mathbb{R}^2 \setminus \{(0,0)\}$ and $\mathcal{U} = [-0.2, 0.2]$, we define the following dynamics and cost functions on the domain $\mathcal{X} \times \mathcal{U}$:

$$f(x, u) = \begin{bmatrix} u & -\omega(x) \\ \omega(x) & u \end{bmatrix} x \quad (\text{B.1})$$

$$l(x, u) = 100\omega(x)^2 + 10 \|x - x_{\text{eq}}\|_2^2 + 1000u^2 \quad (\text{B.2})$$

with $\omega(x) = \|x\|_2^2 - 1$ and $x_{\text{eq}} = [0 \ 1]^\top$.

The system corresponds to clockwise movement inside the unit circle and counterclockwise movement outside of it. The input can be used to reach different rotational speeds to ultimately reach the goal x_{eq} , which is an equilibrium with $u_{\text{eq}} = 0$.

B.2 Flatquad

This is a model of a drone in a two-dimensional world, i.e. a rigid body affected by two input forces and gravity. The control objective is stabilisation to the hovering state with zero velocity, zero position and upwards orientation.

We define a state vector $x = [p_x \ p_y \ s \ c \ v_x \ v_y \ \omega]^\top$ with the following components:

- (p_x, p_y) describing the position of the center of gravity
- $(s, c) \in S^1 \subset \mathbb{R}^2$ as a smooth embedding of the rotation group $\text{SO}(2)$, describing the orientation of the rigid body

B. PROBLEM DEFINITIONS

- (v_x, v_y) describing the velocity of the center of gravity
- Angular velocity ω

The constraint $(s, c) \in S^1$ makes the state space a six-dimensional smooth submanifold of $\mathbb{R}^7$, defined by $0 = m(x) = s^2 + c^2 - 1$.

With an input set $\mathcal{U} := [0, u_{\max}] \times [0, u_{\max}]$ and control input $u = [u_l \ u_r] \in \mathcal{U}$, we define dynamics and cost function:

$$f(x, u) = [v_x \ v_y \ c\omega \ -s\omega \ -s\frac{u_l+u_r}{m} \ c\frac{u_l+u_r}{m} - g \ r\frac{u_l-u_r}{I}]^\top \quad (\text{B.3})$$

$$l(x, u) = \left\| Q^{\frac{1}{2}}(x - x_{\text{eq}}) \right\|_2^2 + \left\| R^{\frac{1}{2}}f(x, u) \right\|_2^2 \quad (\text{B.4})$$

The goal state is $x_{\text{eq}} = [0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0]^\top$, which is an equilibrium with $u_{\text{eq}} = [\frac{mg}{2} \ \frac{mg}{2}]^\top$. We have physical model parameters $m = 20$, $g = 9.81$, $r = 0.5$, $I = m(\frac{r}{2})^2$, and $u_{\max} = 0.6mg$, as well as cost matrices

$$Q^{\frac{1}{2}} = \text{diag}([0.3 \ 0.3 \ \frac{30\pi}{180} \ \frac{30\pi}{180} \ 0.5 \ 0.5 \ \frac{120\pi}{180}])^{-1} \quad (\text{B.5})$$

$$R^{\frac{1}{2}} = \text{diag}([0 \ 0 \ 0 \ 1 \ 1 \ 1]) \quad (\text{B.6})$$

The cost function resembles the classic LQR cost function, but with the input u not penalised directly, but rather through its effect on the dynamics as given by f . Therefore, the R matrix defined here differs from the one typically used in the LQR problem.

Appendix C

Algorithm Parameters

Active learning parameters

Symbol	Parameter Name	Orbits	Flatquad
N_{batch}	Active learning batch size	32	512
σ_{abs}	Absolute uncertainty threshold	0.5	
σ_{rel}	Relative uncertainty threshold	0.05	
α	Kernel output scaling factor	0.5	
η	Relative kernel lengthscale	0.25	
T_{target}	Target timestep	2	1

Loss function parameters

Symbol	Parameter Name	Orbits	Flatquad
w_v	v Sobolev weight	1/11	
w_λ	λ Sobolev weight	10/11	
ε_v	Smallest important v	1	
ε_λ	Smallest important $\ \lambda\ $	1	
δ	λ Huber width	0.3	
μ	Loss fadeout	1	

C. ALGORITHM PARAMETERS

NN & training parameters

Symbol	Parameter Name	Orbits	Flatquad
	Hidden layer sizes	(16, 16, 16)	(128, 128, 128)
	Activation function	jax.nn.softplus	
	Optimiser	optax.adamw [122]	
	Weight Decay	0.0005	
lr_{init}	Initial learning rate	0.01	
lr_{final}	Final learning rate	0.001	
	LR schedule (initial)	Exponential decay	
	LR schedule (curriculum)	Constant (lr_{final})	
	Training epochs (initial)	512	
N_{epochs}	Training epochs (curriculum)	256	
	Minibatch size	32	
N_{ens}	Ensemble size	4	

ODE solver parameters

Symbol	Parameter Name	Orbits	Flatquad
	ODE solver	diffrax.Tsit5 [123]	
	Step size controller	diffrax.PIDController	
$atol$	Absolute error tolerance	10^{-4}	
$rtol$	Relative error tolerance	10^{-4}	
Δt_{min}	Minimum time step	0.01	
Δt_{max}	Maximum time step	0.5	

Table C.1: Algorithm parameters used for the reported results.

Appendix D

Closed-Loop Cost Evaluation

To evaluate our control laws, we define the closed-loop cost

$$V_{\Theta}^{\text{cl}}(x_0) = \int_0^{\infty} l(x(t), u_{\Theta}^*(x)) dt \quad (\text{D.1})$$

with $x(0) = x_0$ and $\dot{x}(t) = f(x(t), u_{\Theta}^*(x(t)))$.

To practically evaluate this quantity, we solve the ODE over a finite time horizon $T_{\text{eval}} = 10$ and add the LQR tail cost to approximate the infinite time horizon. That is, we calculate:

$$V_{\Theta}^{\text{cl}}(x_0) = \int_0^{T_{\text{eval}}} l(x(t), u_{\Theta}^*(x)) dt + V_{\text{LQR}}(x(T_{\text{eval}})) \quad (\text{D.2})$$

with V_{LQR} as defined in A.1. The horizon T_{eval} is selected empirically, such that $x(T_{\text{eval}})$ is close enough to x_{eq} to warrant the LQR approximation.

The cost integral is calculated by augmenting the ODE with an additional state variable v , with $v(0) = 0$ and $\dot{v} = l(x, u_{\Theta}^*(x))$. It is solved numerically with the ODE solver settings given in C, except with ten times smaller tolerances, i.e. $\text{atol} = \text{rtol} = 10^{-5}$.

Trajectory Optimisation Reference Solution

We use `trajax` [124], and specifically its implementation of constrained iLQR `trajax.optimizers.constrained_ilqr` to obtain individual, locally optimal trajectories against which we compare our results. To solve a close approximation of the continuous time problem, we discretise the dynamics with a small timestep of $\Delta t = 0.01$ and a time duration of 5, for a total of 500 time steps. To approximate the infinite horizon problem, we follow the standard practice of using the LQR value function as terminal cost. We ensure empirically that the time horizon is long enough for the terminal state to be very close to the equilibrium, where the LQR value function is a good approximation of V .

The solution given by any trajectory optimiser depends on the initial guess. We use this fact to isolate the distinct local solutions in the vicinity of a nonsmooth point. For some curve $\gamma : [0, 1] \rightarrow \mathcal{X}$, we initialise the trajectory optimiser with the input trajectory given by forward simulation of the approximate optimal control law, starting at $\gamma(0)$ ¹. After obtaining the optimal trajectory, we follow the curve γ in small increments, always using the previous optimal input sequence to produce an initial guess. We obtain the other local solution by starting at $\gamma(1)$ and following the curve in the other direction.

We stress that this procedure only gives us the globally optimal solution if the initial guess given by our approximate optimal feedback law is close enough to the correct local minimum.

¹With the time discretisation used in the trajectory optimiser, rather than the adaptive solver employed otherwise

Appendix F

Parameter Sweeps

To assess the effect of some tuning parameters on the solution, we perform several parameter sweeps. We address the six-dimensional problem defined in B.2, and except for the parameter changed in each sweep the algorithm parameters are as given in C.

For each parameter, we run our algorithm for a range of values. For each single value, the experiment is repeated 4 times with different random seeds to assess repeatability.

For each experiment, we evaluate the following quantities after calculating the control law:

- The relative suboptimality compared to the learned value $\frac{V_{\Theta}^d(x)}{\mu_{\Theta}(x)} - 1$, evaluated at a large set of initial states x sampled according to A.2.
- The relative suboptimality compared to the reference cost $\frac{V_{\Theta}^d(x)}{V_{\text{ref}}(x)} - 1$, evaluated at 256 equidistant points along each curve $\gamma_1, \dots, \gamma_6$ (Fig. 4.6), with the reference V_{ref} calculated as described in E.

Both these quantities are typically spread out over several orders of magnitude. To still convey a useful picture of the results, we plot the fraction of points whose relative suboptimality is below 0.05, 0.5, and 5, respectively.

For each parameter value, we visualise the minimum, mean and maximum of these (empirical) CDF evaluations across the different random seeds with a shaded area plot.

F. PARAMETER SWEEPS

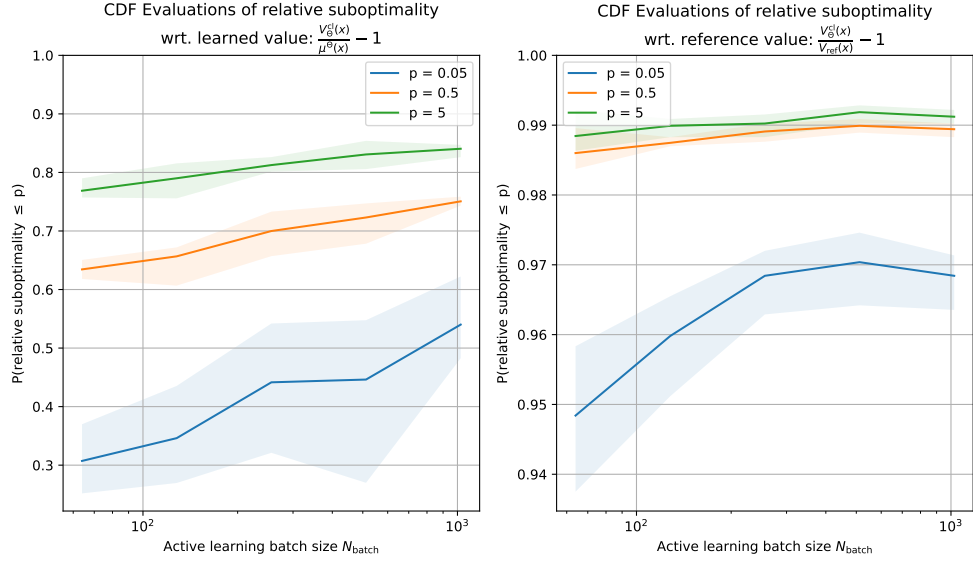


Figure F.1: Active Learning batch size (N_{batch})

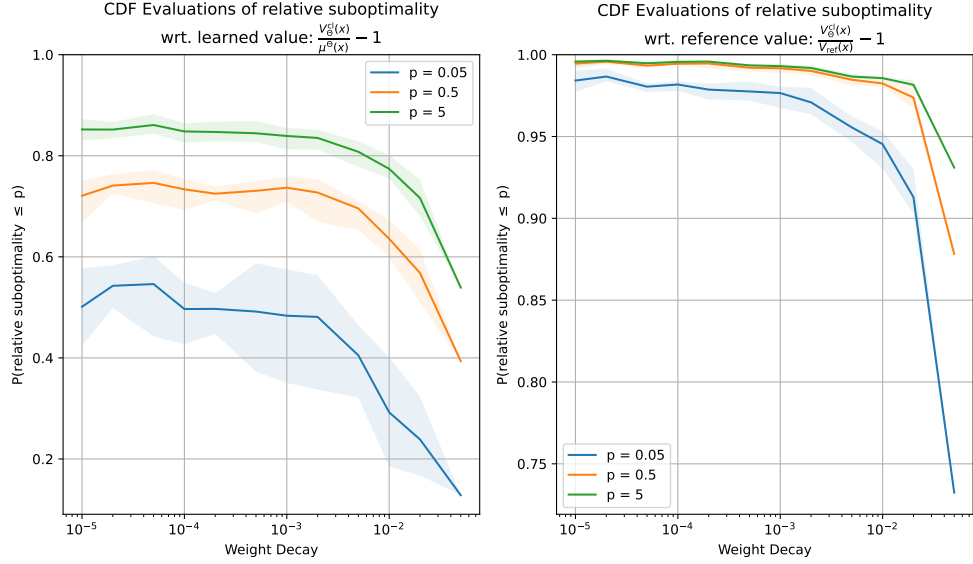


Figure F.2: Weight decay

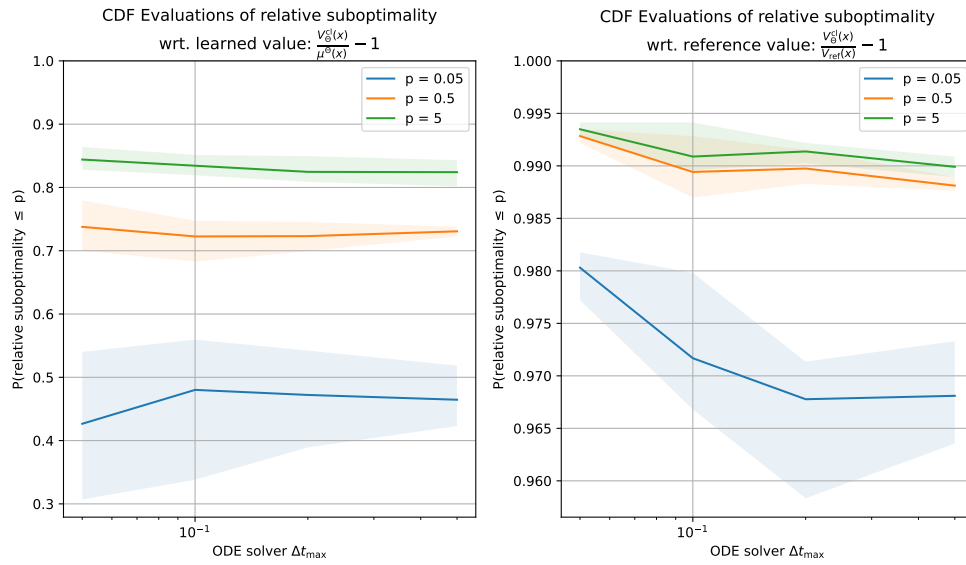


Figure F.3: ODE solver $\Delta t_{\max}$

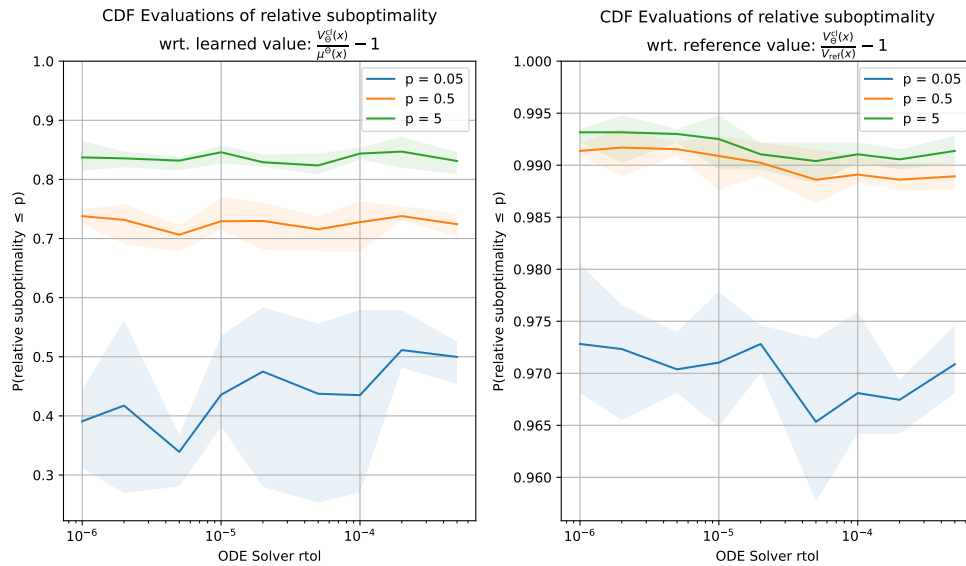


Figure F.4: ODE solver rtol

F. PARAMETER SWEEPS

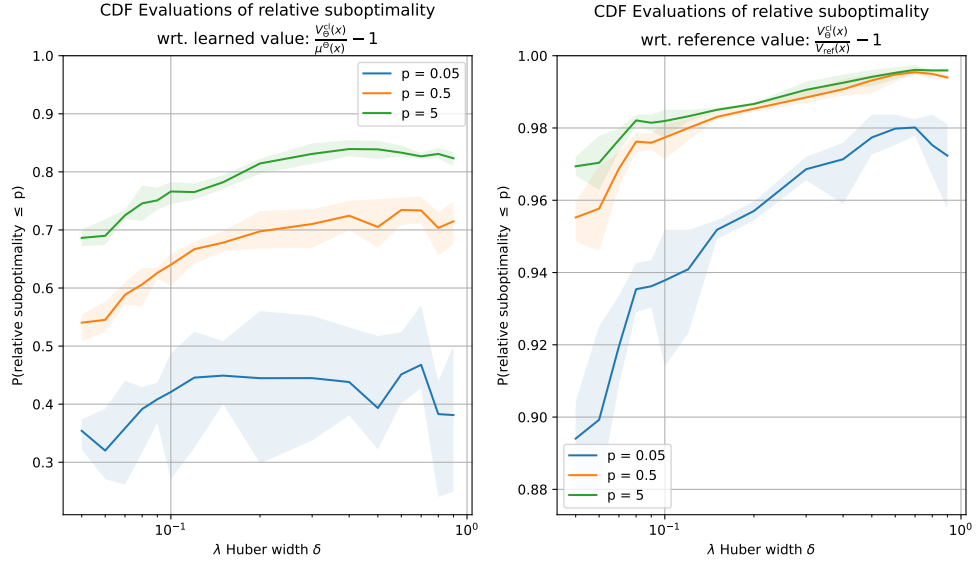


Figure F.5: λ Huber width (δ)

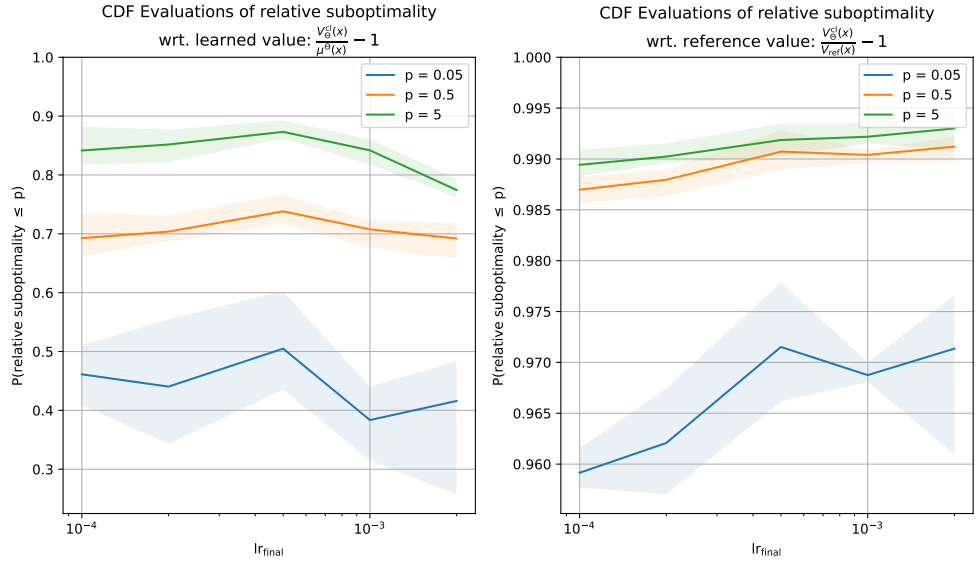


Figure F.6: Final learning rate (lr_{final})

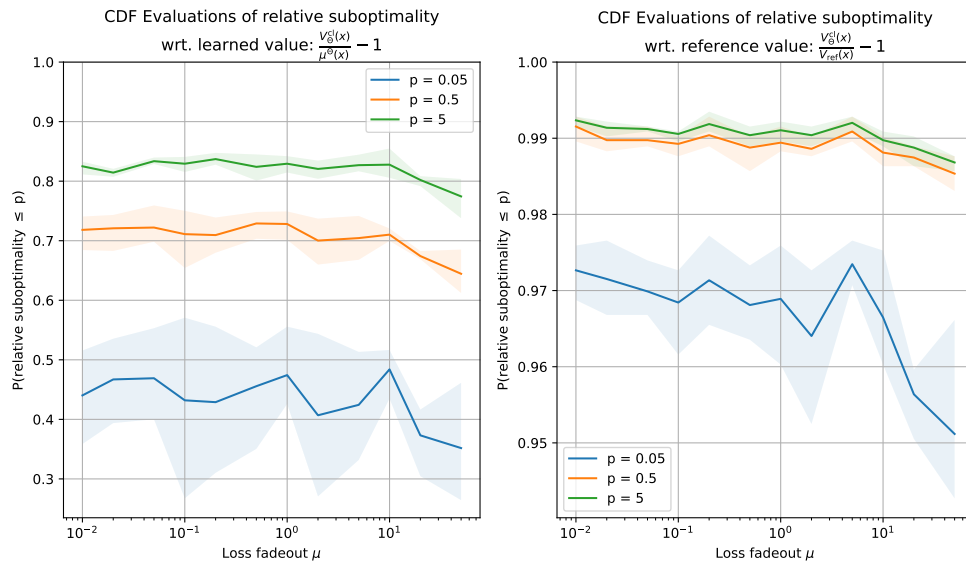


Figure F.7: Loss fadeout (μ)

Bibliography

- [1] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.
- [2] Anil Rao. A survey of numerical methods for optimal control. *Advances in the Astronautical Sciences*, 135, 01 2010.
- [3] S. Karneswaran and L.T. Biegler. Convergence rates for direct transcription of optimal control problems with final-time equality constraints using collocation at radau points. In *2006 American Control Conference*, pages 7 pp.–, 2006.
- [4] J.B. Rawlings, D.Q. Mayne, and M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017.
- [5] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1433–1440, 2016.
- [6] Marin Kobilarov. Cross-entropy motion planning. *The International Journal of Robotics Research*, 31(7):855–871, 2012.
- [7] Carlos Sánchez-Sánchez, Dario Izzo, and Daniel Hennes. Learning the optimal state-feedback using deep networks. In *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1–8, 2016.
- [8] Gao Tang, Weidong Sun, and Kris K. Hauser. Learning trajectories for real-time optimal control of quadrotors. *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3620–3625, 2018.

- [9] Tenavi Nakamura-Zimmerer, Qi Gong, and Wei Kang. Adaptive deep learning for high dimensional Hamilton-Jacobi-Bellman equations. *ArXiv*, abs/1907.05317, 2019.
- [10] Dario Izzo, Ekin Öztürk, and Marcus Mörtens. Interplanetary transfers via deep representations of the optimal policy and/or of the value function. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO '19. ACM, July 2019.
- [11] Guoyuan Chen. Deep neural network approximations for the stable manifolds of the hamilton-jacobi-bellman equations. *IEEE Transactions on Automatic Control*, 2020.
- [12] Behzad Azmi, Dante Kalise, and Karl Kunisch. Data-driven recovery of optimal feedback laws through optimality conditions and sparse polynomial regression. *arXiv: Optimization and Control*, 2020.
- [13] Christoph Zelch, Jan Peters, and Oskar von Stryk. Learning control policies from optimal trajectories. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2529–2535, 2020.
- [14] Karl Kunisch and Daniel Walter. Semiglobal optimal feedback stabilization of autonomous systems via deep neural network approximation, 2020.
- [15] Behzad Azmi, Dante Kalise, and Karl Kunisch. Optimal feedback law recovery by gradient-augmented sparse polynomial regression. *Journal of Machine Learning Research*, 22(48):1–32, 2021.
- [16] Benoit Landry, Hongkai Dai, and Marco Pavone. SEAGuL: Sample efficient adversarially guided learning of value functions. In Ali Jadbabaie, John Lygeros, George J. Pappas, Pablo Parrilo, Benjamin Recht, Claire J. Tomlin, and Melanie N. Zeilinger, editors, *Proceedings of the 3rd Conference on Learning for Dynamics and Control*, volume 144 of *Proceedings of Machine Learning Research*, pages 1105–1117. PMLR, 07 – 08 June 2021.
- [17] Selim Engin and Volkan Isler. Neural optimal control using learned system dynamics, 2023.
- [18] Gianluigi Grandesso, Elisa Alboni, Gastone P. Rosati Papini, Patrick M. Wensing, and Andrea Del Prete. Cacto: Continuous actor-critic with trajectory optimization—towards global optimality. *IEEE Robotics and Automation Letters*, 8(6):3318–3325, June 2023.
- [19] Elisa Alboni, Gianluigi Grandesso, Gastone Pietro Rosati Papini, Justin Carpentier, and Andrea Del Prete. Cacto-sl: Using sobolev learning to improve continuous actor-critic with trajectory optimization, 2023.

-
- [20] Amit Parag, Sébastien Kleff, Léo Saci, Nicolas Mansard, and Olivier Stasse. Value learning from trajectory optimization and Sobolev descent: A step toward reinforcement learning with superlinear convergence properties. In *International Conference on Robotics and Automation (ICRA 2022)*, Philadelphia, United States, May 2022.
 - [21] Miguel Zamora, Roi Poranne, and Stelian Coros. Learning solution manifolds for control problems via energy minimization. *IEEE Robotics and Automation Letters*, 7(3):7912–7919, 2022.
 - [22] Kris Hauser. Learning the problem-optimum map: Analysis and application to global optimization in robotics, 2016.
 - [23] Tomohiro Otsu, Ichiro Maruta, and Kenji Fujimoto. Globally optimal control of nonlinear input-affine systems under quadratic objective functions based on trajectory database. *IFAC-PapersOnLine*, 54(14):37–42, 2021. 3rd IFAC Conference on Modelling, Identification and Control of Nonlinear Systems MICNON 2021.
 - [24] Haoyu Han and Heng Yang. On the nonsmooth geometry and neural approximation of the optimal value function of infinite-horizon pendulum swing-up, 2023.
 - [25] Gao Tang and Kris Hauser. Discontinuity-sensitive optimal control learning by mixture of experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 7892–7898, 2019.
 - [26] Wolfgang Xaver Merkt, Vladimir Ivan, Traiko Dinev, Ioannis Havoutis, and Sethu Vijayakumar. Memory clustering using persistent homology for multimodality- and discontinuity-sensitive learning of optimal control warm-starts. *IEEE Transactions on Robotics*, 37:1649–1660, 2020.
 - [27] Wenjun Zou, Yao Lyu, Jie Li, Yujie Yang, Shengbo Eben Li, Jingliang Duan, Xianyuan Zhan, Jingjing Liu, Yaqin Zhang, and Keqiang Li. Policy bifurcation in safe reinforcement learning. *ArXiv*, abs/2403.12847, 2024.
 - [28] Liyiming Ke, Sanjiban Choudhury, Matt Barnes, Wen Sun, Gilwoo Lee, and Siddhartha Srinivasa. Imitation learning as f -divergence minimization, 2020.
 - [29] Yunzhu Li, Jiaming Song, and Stefano Ermon. Infogail: Interpretable imitation learning from visual demonstrations, 2017.
 - [30] Zhiao Huang, Litian Liang, Zhan Ling, Xuanlin Li, Chuang Gan, and Hao Su. Reparameterized policy learning for multimodal trajectory optimization, 2023.

- [31] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning, 2021.
- [32] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024.
- [33] Zechu Li, Rickmer Krohn, Tao Chen, Anurag Ajay, Pulkit Agrawal, and Georgia Chalvatzaki. Learning multimodal behaviors from scratch with diffusion policy gradient, 2024.
- [34] Richard Bellman. On the theory of dynamic programming. *Proceedings of the National Academy of Sciences*, 38(8):716–719, 1952.
- [35] Daniel Liberzon. *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton University Press, 2011.
- [36] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [37] Yuval Tassa and Tom Erez. Least squares solutions of the hjb equation with neural network value-function approximators. *IEEE Transactions on Neural Networks*, 18(4):1031–1041, 2007.
- [38] Michael Lutter, Boris Belousov, Kim D. Listmann, Debora Clever, and Jan Peters. HJB optimal feedback control with deep differential value functions and action constraints. *ArXiv*, abs/1909.06153, 2019.
- [39] Alena SHILOVA, Thomas Delliaux, Philippe Preux, and Bruno Raffin. Learning HJB viscosity solutions with PINNs for continuous-time reinforcement learning, 2024.
- [40] Tianyu Liu, Steven Ding, Jiarui Zhang, and Liutao Zhou. Pinn-based viscosity solution of hjb equation, 2023.
- [41] Chuwei Wang, Shanda Li, Di He, and Liwei Wang. Is L^2 physics-informed loss always suitable for training physics-informed neural network?, 2022.
- [42] Di He, Shanda Li, Wenlei Shi, Xiaotian Gao, Jia Zhang, Jiang Bian, Liwei Wang, and Tie-Yan Liu. Learning physics-informed neural networks without stacked back-propagation, 2023.

-
- [43] Eloïse Berthier, Justin Carpentier, Alessandro Rudi, and Francis Bach. Infinite-Dimensional Sums-of-Squares for Optimal Control. In *61st IEEE Conference on Decision and Control*, Cancun, Mexico, December 2022. IEEE.
 - [44] Richard Vinter. Convex duality and nonlinear optimal control. *SIAM Journal on Control and Optimization*, 31(2):518–538, 1993.
 - [45] Jean-Bernard Lasserre, Didier Henrion, Christophe Prieur, and Emmanuel Trélat. Nonlinear optimal control via occupation measures and LMI-relaxations, 2007.
 - [46] Jérôme Darbon and Tingwei Meng. On some neural network architectures that can represent viscosity solutions of certain high dimensional hamilton–jacobi partial differential equations. *Journal of Computational Physics*, 425:109907, 2021.
 - [47] Stanley Osher and James A Sethian. Fronts propagating with curvature-dependent speed: Algorithms based on Hamilton-Jacobi formulations. *Journal of Computational Physics*, 79(1):12–49, 1988.
 - [48] Stanley J. Osher and Ronald Fedkiw. *Level set methods and dynamic implicit surfaces.*, volume 153 of *Applied mathematical sciences*. Springer, 2003.
 - [49] Ian M. Mitchell and Yoshihiko Susuki. Level set methods for computing reachable sets of hybrid systems with differential algebraic equation dynamics. In *International Conference on Hybrid Systems: Computation and Control*, 2008.
 - [50] Elizabeth Ann Cross and Ian M. Mitchell. Level set methods for computing reachable sets of systems with differential algebraic equation dynamics. In *2008 American Control Conference*, pages 2260–2265, 2008.
 - [51] Wei Liao, Taotao Liang, Pengwen Xiong, Chen Wang, Aiguo Song, and Peter X. Liu. An improved level set method for reachability problems in differential games, 2022.
 - [52] T. Lolla, M. P. Ueckermann, K. Yiğit, P. J. Haley, and P. F. J. Lermusiaux. Path planning in time dependent flow fields using level set methods. In *2012 IEEE International Conference on Robotics and Automation*, pages 166–173, 2012.
 - [53] Christopher Bohn, Philipp Reis, Manuel Schwartz, and Sören Hohmann. Time and memory-efficient computation of Hamilton-Jacobi reachable sets based on a level set method employing adaptive grids. In 2023

- 62nd IEEE Conference on Decision and Control (CDC), pages 8235–8241, 2023.
- [54] Somil Bansal and Claire Tomlin. Deepreach: A deep learning approach to high-dimensional reachability, 2020.
- [55] Matan Atzmon, Niv Haim, Lior Yariv, Ofer Israelov, Haggai Maron, and Yaron Lipman. Controlling neural level sets, 2019.
- [56] Ishit Mehta, Manmohan Chandraker, and Ravi Ramamoorthi. A level set theory for neural implicit evolution under explicit flows, 2022.
- [57] Dhanushka Kularatne, Subhrajit Bhattacharya, and M. A. Hsieh. Time and energy optimal path planning in general flows. In *Robotics: Science and Systems*, 2016.
- [58] Christian Parkinson and Isabelle Boyle. Efficient and scalable path-planning algorithms for curvature constrained motion in the Hamilton-Jacobi formulation. *ArXiv*, abs/2304.12377, 2023.
- [59] Bastien Schnitzler, Antoine Drouin, Jean-Marc Moschetta, and Daniel Delahaye. General extremal field method for time-optimal trajectory planning in flow fields. *IEEE Control Systems Letters*, 7:2605–2610, 2023.
- [60] James P. England, Bernd Krauskopf, and Hinke M. Osinga. Computing two-dimensional global invariant manifolds in slow-fast systems. *Int. J. Bifurc. Chaos*, 17:805–822, 2007.
- [61] Blane Rhoads, Igor Mezić, and Andrew C. Poje. Minimum time heading control of underpowered vehicles in time-varying ocean currents. *Ocean Engineering*, 66:12–31, 2013.
- [62] Andrei Marchidan and Efstathios Bakolas. Numerical techniques for minimum-time routing on sphere with realistic winds. *Journal of Guidance Control and Dynamics*, 39:188–193, 2016.
- [63] Hinke M. Osinga and John Hauser. The geometry of the solution set of nonlinear optimal control problems. *Journal of Dynamics and Differential Equations*, 18(4):881–900, Oct 2006.
- [64] Thomas Holzhüter. Optimal regulator for the inverted pendulum via Euler–Lagrange backward integration. *Automatica*, 40(9):1613–1620, 2004.
- [65] Thomas Holzhüter and Thomas Klinker. Method to solve the nonlinear infinite horizon optimal control problem with application to the track

- control of a mobile robot. *International Journal of Bifurcation and Chaos*, 17(10):3607–3611, 2007.
- [66] Meir Pachter and Isaac E Weintraub. The synthesis of optimal control laws using isaacs’ method for the solution of differential games, 2023.
- [67] Takamasa Horibe and Noboru Sakamoto. Optimal swing up and stabilization control for inverted pendulum via stable manifold method. *IEEE Transactions on Control Systems Technology*, 26(2):708–715, 2018.
- [68] Guoyuan Chen and Gaosheng Zhu. Symplectic algorithms for stable manifolds in control theory, 2021.
- [69] Lawrence C. Evans. *Partial differential equations*. American Mathematical Society, Providence, R.I., 2010.
- [70] Alberto Bressan. Viscosity solutions of Hamilton-Jacobi equations and optimal control problems, 2011.
- [71] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. Mathematical theory of optimal processes, 1962.
- [72] Erkut Aykutlug, Ufuk Topcu, and Kenneth D. Mease. Manifold-Following Approximate Solution of Completely Hypersensitive Optimal Control Problems. *Journal of Optimization Theory and Applications*, 170(1):220–242, July 2016.
- [73] Arthur E. Bryson, Yu-Chi Ho, and George M. Siouris. Applied optimal control: Optimization, estimation, and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(6):366–367, 1979.
- [74] Moritz Diehl. Numerical optimal control. 2011.
- [75] John M. Lee. *Smooth Manifolds*. Springer New York, New York, NY, 2003.
- [76] Yuri A. Kuznetsov. *Elements of applied bifurcation theory*. Number 112 in Applied Mathematical Sciences. Springer, Berlin, 2 edition, 1998.
- [77] Nina N. Subbotina and Timofey B. Tokmantsev. Classical characteristics of the bellman equation in constructions of grid optimal synthesis. *Proceedings of the Steklov Institute of Mathematics*, 271:246–264, 2010.
- [78] Nathalie Caroff and Halina Frankowska. Conjugate points and shocks in nonlinear optimal control. *Transactions of the American Mathematical Society*, 348:3133–3153, 1996.

- [79] Dieter Grass, Jonathan P. Caulkins, Gustav Feichtinger, Gernot Tragler, and Doris A. Behrens. *Optimal Control of Nonlinear Processes*. Number 978-3-540-77647-5 in Springer Books. Springer, December 2008.
- [80] S. Moghayer and F.O.O. Wagener. Genesis of indifference thresholds and infinitely many indifference points in discrete time infinite horizon optimisation problems. CeNDEF Working Papers 09-14, Universiteit van Amsterdam, Center for Nonlinear Dynamics in Economics and Finance, 2009.
- [81] Moiseev, Igor and Sachkov, Yuri L. Maxwell strata in sub-riemannian problem on the group of motions of a plane. *ESAIM: COCV*, 16(2):380–399, 2010.
- [82] Erik J. Bekkers, Remco Duits, Alexey Pavlovich Mashtakov, and Gonzalo Sanguinetti. A PDE approach to data-driven sub-riemannian geodesics in $SE(2)$. *SIAM J. Imaging Sci.*, 8:2740–2770, 2015.
- [83] Dong Eui Chang. A simple proof of the pontryagin maximum principle on manifolds. *Automatica*, 47(3):630–633, 2011.
- [84] Andersen Ang. Coercive functions. Lecture Notes, 2023.
- [85] Burr Settles. Active learning literature survey. *University of Wisconsin, Madison*, 52, 2010.
- [86] Yuxin Chen and Andreas Krause. Near-optimal batch mode active learning and adaptive submodular optimization. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 160–168, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR.
- [87] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach, 2018.
- [88] Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. Batchbald: Efficient and diverse batch acquisition for deep Bayesian active learning, 2019.
- [89] David Holzmüller, Viktor Zaverkin, Johannes Kästner, and Ingo Steinwart. A framework and benchmark for deep batch active learning for regression, 2023.
- [90] Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. Safe exploration for optimization with Gaussian processes. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on*

- Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 997–1005, Lille, France, 07–09 Jul 2015. PMLR.
- [91] Rikky R.P.R. Duivenvoorden, Felix Berkenkamp, Nicolas Carion, Andreas Krause, and Angela P. Schoellig. Constrained Bayesian optimization with particle swarms for safe adaptive controller tuning. *IFAC-PapersOnLine*, 50(1):11800–11807, 2017. 20th IFAC World Congress.
 - [92] Christian Fiedler, Johanna Menn, Lukas Kreisköther, and Sebastian Trimpe. On safety in safe Bayesian optimization, 2024.
 - [93] Wojciech M. Czarnecki, Simon Osindero, Max Jaderberg, Grzegorz Swirszcz, and Razvan Pascanu. Sobolev training for neural networks. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
 - [94] H. Drucker and Y. Le Cun. Improving generalization performance using double backpropagation. *IEEE Transactions on Neural Networks*, 3(6):991–997, 1992.
 - [95] Ryusuke Masuoka, Sebastian Thrun, and Tom M Mitchell. Constraining neural networks to fit target slopes. 1993.
 - [96] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4:251–257, 1991.
 - [97] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2006.
 - [98] E. Solak, Roderick Murray-Smith, William E. Leithead, Douglas J. Leith, and Carl Edward Rasmussen. Derivative observations in Gaussian process models of dynamic systems. In *NIPS*, 2002.
 - [99] Max Welling and Yee Whye Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, page 681–688, Madison, WI, USA, 2011. Omnipress.
 - [100] Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic gradient descent as approximate Bayesian inference. *Journal of Machine Learning Research*, 18(134):1–35, 2017.
 - [101] Pavel Izmailov, Sharad Vikram, Matthew D Hoffman, and Andrew Gordon Gordon Wilson. What are Bayesian neural network posteriors really like? In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of

- Proceedings of Machine Learning Research*, pages 4629–4640. PMLR, 18–24 Jul 2021.
- [102] Mohammad Emtiyaz E Khan, Alexander Immer, Ehsan Abedi, and Maciej Korzepa. Approximate inference turns deep networks into Gaussian processes. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [103] Julyan Arbel, Konstantinos Pitas, Mariia Vladimirova, and Vincent Fortuin. A primer on Bayesian neural networks: Review and debates, 2023.
- [104] Eduardo Sontag and Hector Sussmann. Remarks on continuous feedback. In *1980 19th IEEE Conference on Decision and Control including the Symposium on Adaptive Processes*, pages 916–921, 1980.
- [105] Eduardo D. Sontag. Stability and stabilization: Discontinuities and the effect of disturbances, 1999.
- [106] Karl Kunisch and Donato Vásquez-Varas. Smooth approximation of feedback laws for infinite horizon control problems with non-smooth value functions, 2024.
- [107] R. Bellman, Rand Corporation, and Karreman Mathematics Research Collection. *Dynamic Programming*. Rand Corporation research study. Princeton University Press, 1957.
- [108] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume I. Athena Scientific, Belmont, MA, USA, 3rd edition, 2005.
- [109] Richard F. Hartl, Suresh P. Sethi, and Raymond G. Vickson. A survey of the maximum principles for optimal control problems with state constraints. *SIAM Review*, 37(2):181–218, 1995.
- [110] R. Vinter. *Optimal Control*. Systems & Control: Foundations & Applications. Birkhäuser Boston, 2000.
- [111] R. Isaacs. *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. Dover books on mathematics. Wiley, 1965.
- [112] Dustin Kenefake and Efstratios N Pistikopoulos. Ppopt-multiparametric solver for explicit mpc. In *Computer Aided Chemical Engineering*, volume 51, pages 1273–1278. Elsevier, 2022.

-
- [113] M. Kvasnica, P. Grieder, and M. Baotić. Multi-Parametric Toolbox (MPT), 2004.
- [114] Chantal Landry, Alexandre Caboussat, and Ernst Hairer. Solving optimization-constrained differential equations with discontinuity points, with application to atmospheric chemistry. *SIAM Journal on Scientific Computing*, 31(5):3806–3826, 2009.
- [115] Taylor A. Howell, Simon Le Cleac’h, Sumeet Singh, Pete Florence, Zachary Manchester, and Vikas Sindhvani. Trajectory optimization with optimization-based dynamics, 2023.
- [116] Russ Tedrake. *Underactuated Robotics*. 2023.
- [117] P. Charbonnier, L. Blanc-Feraud, G. Aubert, and M. Barlaud. Deterministic edge-preserving regularization in computed imaging. *IEEE Transactions on Image Processing*, 6(2):298–311, 1997.
- [118] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [119] Ernst Hairer. Solving differential equations on manifolds, 2011.
- [120] Anders Forsgren, Philip E. Gill, and Elizabeth Wong. Primal and dual active-set methods for convex quadratic programming. *Mathematical Programming*, 159(1–2):469–508, December 2015.
- [121] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004.
- [122] DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Claudio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel, Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch, Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec, Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020.
- [123] Patrick Kidger. *On Neural Differential Equations*. PhD thesis, University of Oxford, 2021.

BIBLIOGRAPHY

- [124] Stephen Tu, Ray Frostig, Sumeet Singh, and Vikas Sindhwani. Trajax: A Python library for differentiable optimal control on accelerators., 2021.

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Continuous-Time Approximate Dynamic Programming
as an Active Learning Problem

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

Dettling

First name(s):

Mattia Balduin Julian

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Zurich, 2024-07-02

Signature(s)



For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.